

동시성을 위한 Go

5편: 대규모 동시성

남궁명수

[5] 대규모 동시성	3
[5-1] 에러 전파(Error Propagation)	3
(1) 에러 전파 철학: 에러는 일급 관리 대상이다.	3
(2) 컴포넌트 경계면에서의 에러 래핑 원칙	5
(3) 에러 전파 프레임워크 전체 소스코드 구현	6
(4) 결론	11
[5-2] 타임아웃과 취소(Timeouts and Cancellation)	12
(1) 시스템에 타임아웃을 반드시 심어야 하는 3가지 이유	12
(2) 연산 중단 능력: 프리엠션(선점)의 정석	13
(3) 공유 상태 오염과 트랜잭션 원자적 쓰기 최적화	15
(4) 중단 도미노 현상과 중복 데이터 수신 문제(Duplicate Message)	16
(5) 결론	17
[5-3] 하트비트(Heartbeats)	18
(1) 일정 시간 간격으로 뛰는 하트비트(Interval-Based Heartbeats)	18
(2) 하트비트를 통한 고루틴 건강 상태 진단과 데드락 방지	21
(3) 단위 작업 시작 기반의 하트비트(Work-Based Heartbeats)	23
(4) 비동기 테스트의 난제 극복: 결정성 100% 테스트 설계	25
(5) 결론	28
[5-4] 복제된 요청(Replicated Requests)	29
(1) 복제된 요청 패턴 전체 소스코드 구현	29
(2) 최악의 타이밍 병목 파괴와 속도 기적의 원리	31
(3) 실무 아키텍처 설계 시 반드시 지켜야 할 ‘동등성’과 ‘균일성 파괴’법칙	32
(4) 결론	32
[5-5] 처리량 제한(Rate Limiting)	33
(1) 시스템에 처리량 제한을 반드시 심어야 하는 이유	33
(2) 토큰 버킷 알고리즘의 원리	34
(3) Go 표준 라이브러리를 활용한 단일 레이트 리미터 구현	34
(4) 다중 계층 복합 처리량 제한 패턴	38
(5) 시간, 디스크, 네트워크를 넘나드는 다차원 자원 제한 설계	43
(6) 결론	52
[5-6] 비정상 고루틴 자가 치유(Healing Unhealthy Goroutines)	53
(1) 관리자와 피관리자: 스튜어드와 워드 아키텍처	53
(2) 무책임한 일꾼 고루틴 저격 테스트	56
(3) 클로저와 채널 브리징을 통한 실무형 복잡 데이터 스트림 결합 기술	58
(4) 자가 치유 아키텍처 가동 시 주의해야 할 상태 보존 법칙	63
(5) 결론	64

[5] 대규모 동시성

이제 Go에서 동시성을 활용하는 몇 가지 일반적인 패턴을 배웠으니, 이제 이러한 패턴들을 조합하여 대규모로 확장 가능한 시스템을 작성할 수 있게 해주는 일련의 실천 방법에 대해 살펴보겠다.

이 장에서는

- 단일 프로세스 내부에서 동시 작업을 확장하는 방법
- 그리고 여러 프로세스를 다룰 때 동시성이 어떻게 적용되는지

에 대해 논의할 것이다.

[5-1] 에러 전파(Error Propagation)

대규모 복잡한 시스템이나 고성능 동시성 프로그래밍 환경에서 시스템의 신뢰성을 결정짓는 가장 중요한 아키텍처 중 하나인 에러 전파 철학과 설계 프레임워크에 대해 알아보겠다.

동시성 코드를 작성할 때, 특히 여러 장비가 네트워크로 얹힌 분산 시스템 환경에서는 무언가 잘못되기가 매우 쉽고, 반대로 “도대체 왜 에러가 났는지” 그 근본 원인을 추적하기는 지옥 난이도로 어렵다.

에러가 내 시스템 내부망을 어떻게 타고 흐르며, 최종 사용자에게 어떤 모습으로 노출될 것인지를 정교하게 설계해 두지 않으면 운영 단계에서 엄청난 고통을 받게 된다.

4장의 에러 처리 섹션에서는 고루틴 내부의 에러를 채널을 통해 부모에게 전달하는 ‘방법’을 배웠지만, 그 에러의 ‘알맹이 스펙’이 어떻게 생겨야 하고 거대한 아키텍처 경계를 어떻게 타고 넘어가야 하는지에 대해서는 다루지 않았다. 이번장에서는 시스템의 통제력을 100% 유지할 수 있는 독창적인 에러 전파 철학을 명확하게 마스터해보자.

(1) 에러 전파 철학: 에러는 일급 관리 대상이다.

많은 개발자가 저지르는 가장 흔한 실수는 에러 전파를 시스템의 메인 데이터 흐름 뒤에 뒤따라오는 ‘부차적이고 귀찮은 찌꺼기’ 정도로 취급하는 것이다.

데이터가 흐르는 파이프라인 아키텍처는 밤새워 아름답게 설계하면서도, 에러는 그저 마지못해 허용하다가 스택 위로 대책 없이 던져 올린 뒤 최종 사용자의 화면에 가차 없이 날것 그대로 배설(Dump)해 버리곤 한다.

Go 언어는 호출 스택의 매 프레임마다 개발자가 직접 에러를 처리하도록 문법적으로 강제함으로 써 이 나쁜 관행을 바로잡으려고 노력했다.

하지만 여전히 실무 코드에서 에러는 시스템의 제어 흐름 뒤편에 방치되어 있다. 아주 약간의 사전 설계와 최소한의 오버헤드만 투자하면 에러 처리는 내 시스템의 강력한 자산이자 사용자 경험의 핵심 무기가 될 수 있다.

먼저 본질적인 질문을 던져보겠다.

“에러란 무엇이며, 왜 발생하고, 어떤 가치가 있는 정보를 담고 있어야 하는가?”

에러란

내 시스템이 사용자가 명시적으로 혹은 암묵적으로 요청한 특정 연산을 완벽하게 완수할 수 없는 예외 상태에 진입했음을 뜻한다. 따라서 아키텍처 관점에서 에러는 상류의 책임자들에게 다음 4가지 크리컬 정보를 반드시 배달해야 한다.

- 무슨 일이 일어났는가

에러의 원인이 되는 명확한 핵심 요약 정보이다.
(예: “디스크 용량 가득참”, “네트워크 소켓 닫힘”, “인증 자격 증명 만료” 등)

- 언제 어디서 발생했는가

에러가 최초로 인스턴스화된 최하단 스택 위치부터 시작하여 최상위 지점까지 ‘완벽한 스택 추적 (Stack Trace)’ 정보가 포함되어야 한다.
또한 분산 시스템 환경이라면 에러가 터진 물리적인 장비의 식별 정보(Machine ID 혹은 IP)와 정확한 UTC 기준의 타임스탬프 시각이 박혀있어야 사후 추적이 가능해진다.

- 사용자 친화적인 인간 중심 메시지

시스템 내부의 거친 하드웨어 에러 메시지를 가공하여 사용자가 이해할 수 있는 정돈된 한 줄 짜리 인간 중심 언어로 출력해야 한다. 이 문제가 일시적인 순간 장애인지 아닌지도 힌트를 주어야 한다.

- 상세 조사를 위한 통합 로그 식별자

사용자가 에러 화면을 마주했을 때, 기술 지원 팀에 문의하거나 추적할 수 있는 고유한 ‘로그 ID(GUID 혹은 일련번호)’를 화면에 제공해야 한다.
이 ID는 서버 시스템의 중앙 집중형 로그 저장소에 박힌 스택 추적, 타임스탬프 등의 방대한 상세 로우 데이터와 1:1로 매핑되는 크로스 참조 열쇠 역할을 수행한다.

따라서 우리는 에러 전파 프레임워크를 설계할 때 내 시스템의 모든 에러를 다음 두 가지 카테고리로 엄격하게 분류하는 관점을 지녀야 한다.

1. 정상적이고 잘 짜인 ‘알려진 예외 케이스’

네트워크 단절, 디스크 쓰기 실패 등 우리 시스템 스펙 내에서 완벽하게 커스텀 가공되어 제어 흐름을 타는 에러

2. 설계되지 않은 생 날 것의 에러 즉, 버그(Bugs)

위 4가지 필수 아키텍처 정보가 누락된 채 외부 라이브러리나 OS 밑바닥에서 직통으로 뿜어져 올라와 가공 없이 유통되는 모든 비정상적 에러

(2) 컴포넌트 경계면에서의 에러 래핑 원칙

거대한 마이크로서비스나 다중 레이어 모듈 시스템을 구축할 때 이 철학은 엄청난 위력을 발휘한다. 아래와 같이 3개의 컴포넌트 레이어가 직렬로 연결된 대형 시스템이 있다고 가정해보자.

최하단의 '저수준 컴포넌트'에서 완벽한 스택 추적 정보가 담긴 이쁜 정석 에러가 발생하여 위로 전달되었다. 저수준 컴포넌트 내부 관점에서 정석 에러가 맞지만, 이를 넘겨 받은 '중간 매개 컴포넌트' 관점에서는 이것도 가공되지 않은 날 것의 외부 에러일 뿐이다.

따라서 동시성 아키텍처의 대원칙은 다음과 같다.

“구조 체인의 결합도 격리를 위해, 각 모듈 컴포넌트의 경계면(Public 메서드/함수 인터페이스)을 통과하는 모든 인입 에러들은 반드시 해당 모듈 고유의 에러 타입으로 묶어주는 '에러 래핑'을 강제로 수행해야 한다.”

이 원칙을 컴포넌트 길목에 적용하는 정석적인 구조 레이아웃 코드를 확인해보자.

```
func PostReport(id string) error {
    result, err := lowlevel.DoWork()
    if err != nil {
        // 길목 체크: 넘어온 에러가 하위 레이어의 정석 에러 스펙이 맞는지 검증합니다.
        // 만약 이 검증이 실패하면 가공되지 않은 기습 버그 에러로 취급되어 상류로
        ferry(수송)됩니다.
        if _, ok := err.(lowlevel.Error); ok {
            // 현재 우리 '중간 매개 모듈'의 비즈니스 컨텍스트에 맞게 고유 타입으로
            에러를 이쁘게 포장(Wrap)합니다.
            // 이 과정에서 최종 사용자에게 노출할 필요가 없는 저수준의 구체적인
            하드웨어 세부 세부사항은 Masking(은폐) 처리합니다.
            err = WrapErr(err, "cannot post report with id %q", id)
        }
        return err
    }
    // ... 비즈니스 로직 계속 진행
}
```

최초로 에러가 인스턴스화되었던 밑바닥의 구체적인 하드웨어 정보와 고루틴 스택 위치는 내부 주머니에 온전히 보존된 채, 모듈의 경계를 넘어갈 때마다 껍데기가 현재 모듈의 우아한 에러 타입으로 변환되는 구조이다.

이렇게 설계하면 우리 모듈의 통제 영역을 벗어나 탈출하는 에러 중 내 고유 타입으로 포장되지 않은 모든 놈들을 명확하게 '말형성된 에러, 즉 버그'로 인지할 수 있게 된다. 이 사상을 바탕으로 코드를 짜면 시스템의 에러 무결성이 유기적으로 자라나는 발현적 속성을 얻게 되며, 시간이 흐를수록 시스템이 더 단단하고 견고하게 진화하는 프레임워크를 확보할 수 있다.

(3) 에러 전파 프레임워크 전체 소스코드 구현

이제 이 에러 전파 프레임워크의 사상을 정직하게 구현하여 버그 에러와 정석 커스텀 에러가 어떻게 다르며 판별되고 유통되는지 생략없는 전체 코드로 검증해보자.

1단계 상황: 중간 레이어에서 에러 레핑을 누락했을 때의 대참사

하위 모듈의 날 것 에러가 그대로 메인 유저 화면까지 유출되어 버그로 감지되는 전체 소스코드이다.

```
package main

import (
    "fmt"
    "log"
    "os"
    "os/exec"
    "runtime/debug"
)

// 모든 정석 에러의 정보를 품고 움직일 마스터 에러 구조체 정의
type MyError struct {
    Inner error          // 우리가 감싸안은 내부 원본 에러 정보 (최하단 원인 추적
                        // 용)
    Message string        // 상류 및 사용자가 보게 될 가공된 에러 메시지
    StackTrace string     // 에러가 태어난 그 물리적 순간의 완벽한 고루틴 스택 기록
    Misc map[string]interface{} // 고루틴 ID, 장비명, 스택 해시 등 온갖 컨텍스트 정보를 담을 주머니
}

func wrapError(err error, messagef string, msgArgs ...interface{}) MyError {
    return MyError{
        Inner:    err,
        Message:  fmt.Sprintf(messagef, msgArgs...),
        StackTrace: string(debug.Stack()), // 핵심: 에러 인스턴스가 생성되는 정밀한 타임밍의 스택 추적을 자동 박멸하여 기록
        Misc:     make(map[string]interface{}),
    }
}

func (err MyError) Error() string {
    return err.Message
}

// =====
// 1. 최하단 저수준 컴포넌트 레이어 (lowlevel)
// =====
type LowLevelErr struct {
    error
}

func isGloballyExec(path string) (bool, error) {
    info, err := os.Stat(path)
```

```

        if err != nil {
            // os.Stat이 던진 거친 날것의 에러를 자신 고유의 LowLevelErr 정석 스펙으로
            포장하여 전달
            return false, LowLevelErr{wrapError(err, err.Error())}
        }
        return info.Mode().Perm() & 0100 == 0100, nil
    }

// =====
// 2. 중간 매개 컴포넌트 레이어 (intermediate)
// =====
type IntermediateErr struct {
    error
}

func runJob(id string) error {
    const jobBinPath = "/bad/job/binary"
    isExecutable, err := isGloballyExec(jobBinPath)
    if err != nil {
        // 설계적 대실수 지점: 하위 모듈이 준 에러를 자신의 고유 타입(IntermediateErr)
        으로
        // 래핑하지 않고 상류로 날것 그대로 통과(Ferry)시켜 버렸습니다.
        return err
    } else if !isExecutable {
        return wrapError(nil, "job binary is not executable")
    }
    return exec.Command(jobBinPath, "--id="+id).Run()
}

// =====
// 3. 최상위 사용자 대면 레이어 (main 및 핸들러)
// =====
func handleError(key int, err error, message string) {
    // 로그 식별 열쇠 ID인 logID와 상세 로우 에러 정보를 안전하게 매핑하여 서버 통합 데
    이터로그에 저장
    log.SetPrefix(fmt.Sprintf("[logID: %v]: ", key))
    log.Printf("%#v", err)

    // 최종 사용자의 브라우저나 stdout 화면에는 정제된 안전한 메시지만 송출
    fmt.Printf("[%v] %v\n", key, message)
}

func main() {
    log.SetOutput(os.Stdout)
    log.SetFlags(log.Ltime | log.LUTC)

    err := runJob("1")
    if err != nil {
        // 기본적으로 정제 불명의 말형성된 에러(Bug)가 올라왔을 때를 대비한 인간 중
        심 방어형 메시지 세팅
        msg := "There was an unexpected issue; please report this as a bug."
    }
}

```

```

// 엄격한 관문 판별: 내가 기대한 내 바로 하위 레이어의 정석 에러 타입이 맞는
지 체크함
if _, ok := err.(IntermediateErr); ok {
    msg = err.Error() // 내 하위 정석 에러가 맞을 때만 안심하고 그 커스텀
에러 문구를 노출함
}

handleError(1, err, msg)
}
}

```

위 코드를 가동하여 실행하면 서버 로그와 사용자 화면(stdout)에 각각 다음과 같은 격리된 결과가 출력된다.

```

// 1. 서버 내부 중앙 집중형 데이터로그 저장소에 기록되는 정밀 추적 로그 데이터:
[logID: 1]: 21:46:07 main.LowLevelErr{error:main.MyError{Inner:(*os.PathError)
(0xc4200123f0),
Message:"stat /bad/job/binary: no such file or directory",
StackTrace:"goroutine 1 [running]:Wnruntime/debug.Stack(0xc420012420, 0x2f,
0xc420045d80)WnWt/src/runtime/debug/stack.go:24 +0x79Wnmain.wrapError(0x530200,
0xc4200123f0, 0xc420012420, 0x2f, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, ...)WnWt/tmp/
main.go:22 +0x62Wnmain.isGloballyExec(0x4d1313, 0xf, 0xc420045eb8, 0x487649,
0xc420056050)WnWt/tmp/main.go:37 +0xaaWnmain.runJob(0x4cfada, 0x1, 0x4d4c35,
0x22)WnWt/tmp/main.go:47 +0x48Wnmain.main()WnWt/tmp/main.go:67 +0x63Wn",
Misc:map[string]interface {}{}}

// 2. 최종 사용자 화면에 송출되는 안전한 인간 중심 메시지:
[1] There was an unexpected issue; please report this as a bug.

```

결과 지표 화면을 보자.

에러 전파 경로 중간에 있는 intermediate 모듈 경계면에서 래핑 규칙을 위반하고 하위 에러를 날 것으로 방치했기 때문에, 최상단 main 관문 검사 기구(err.(IntermediateErr))에서 이 에러를 위험한 미설계 ‘버그’로 명확하게 분류해 냈다.

그리고 시스템은 혹시 모를 거친 소스코드 주소나 해킹 단서가 될 만한 시스템 정보 유출을 원천 차단하기 위해, 사용자의 화면에는 단호하고 깔끔한 인간 중심 방어형 버그 리포트 문구만 보여주고 상세 내역은 로그 서버 인프라 속으로 격리 수용했다.

에러의 신뢰성 등급이 무너졌을 때 시스템이 스스로를 방어하는 완벽한 메커니즘이다.

2단계 상황: 중간 레이어의 에러 래핑 규칙을 올바르게 수리했을 때
이제 중간 컴포넌트인 runJob 함수가 자신의 모듈 경계면 책임을 다하도록 고유 타입 래핑
방어벽 코드를 안전하게 보수하겠다.

```
package main

import (
    "fmt"
    "log"
    "os"
    "os/exec"
    "runtime/debug"
)

type MyError struct {
    Inner      error
    Message    string
    StackTrace string
    Misc       map[string]interface{}
}

func wrapError(err error, messagef string, msgArgs ...interface{}) MyError {
    return MyError{
        Inner:      err,
        Message:    fmt.Sprintf(messagef, msgArgs...),
        StackTrace: string(debug.Stack()),
        Misc:       make(map[string]interface{}),
    }
}

func (err MyError) Error() string {
    return err.Message
}

type LowLevelErr struct {
    error
}

func isGloballyExec(path string) (bool, error) {
    info, err := os.Stat(path)
    if err != nil {
        return false, LowLevelErr{wrapError(err, err.Error())}
    }
    return info.Mode().Perm() & 0100 == 0100, nil
}

// =====
// 2. 중간 매개 컴포넌트 레이어 (정식 수리 완료)
// =====
type IntermediateErr struct {
    error
}
```

```

func runJob(id string) error {
    const jobBinPath = "/bad/job/binary"
    isExecutable, err := isGloballyExec(jobBinPath)
    if err != nil {
        // 정석 최적화 수리 완료: 하위 모듈 에러를 가만히 두지 않고,
        // 자신 모듈의 명확한 비즈니스 컨텍스트 메시지를 심어 IntermediateErr 고유 타
        입으로 단단히 밀봉(Wrap)합니다.
        return IntermediateErr{wrapError(
            err,
            "cannot run job %q: requisite binaries not available",
            id,
        )}
    } else if isExecutable == false {
        return IntermediateErr{wrapError(
            nil,
            "cannot run job %q: requisite binaries are not executable",
            id,
        )}
    }
    return exec.Command(jobBinPath, "--id="+id).Run()
}

func handleError(key int, err error, message string) {
    log.SetPrefix(fmt.Sprintf("[logID: %v]: ", key))
    log.Printf("%#v", err)
    fmt.Printf("[%v] %v\n", key, message)
}

func main() {
    log.SetOutput(os.Stdout)
    log.SetFlags(log.Ltime | log.LUTC)

    err := runJob("1")
    if err != nil {
        msg := "There was an unexpected issue; please report this as a bug."

        // 관문 판별 통과: 이제 내 하위 정석 에러 타입이 맞음이 100% 컴파일 타임에
        입증되었습니다.
        if _, ok := err.(IntermediateErr); ok {
            msg = err.Error() // 안심하고 정성스레 가공된 비즈니스 힌트 메시지를
            유저에게 송출합니다.
        }

        handleError(1, err, msg)
    }
}

```

수리 완료된 코드를 가동하여 실행하면 로그 저장소와 사용자 화면에 각각 다음과 같은 결과가 출력된다.

```
// 1. 로그 서버 인프라에 쌓이는 더욱 풍부해진 다중 계층 양파 래핑 로그 텍스트:
[logID: 1]: 22:11:04
main.IntermediateErr{error:main.MyError{Inner:main.LowLevelErr{error:main.MyError{Inner:(*os.PathError)(0xc4200123f0), Message:"stat /bad/job/binary: no such file or directory", StackTrace:"goroutine 1 [running]:Wn... 생략 ...Wn", Misc:map[string]interface {}{}}, Message:"cannot run job W"1W": requisite binaries not available", StackTrace:"goroutine 1 [running]:Wnruntime/debug.Stack(0x4d63f0, 0x33, 0xc420045e40)WnWt/src/runtime/debug/stack.go:24 +0x79Wnmain.wrapError(0x530380, 0xc42000a330, ...)WnWt/tmp/main.go:53 +0x356Wnmain.main()WnWt/tmp/main.go:71 +0x63Wn", Misc:map[string]interface {}{}}}

// 2. 최종 사용자 화면에 송출되는 완벽하게 통제된 우아한 안내 문구:
[1] cannot run job "1": requisite binaries not available
```

아키텍처 최적화 결과를 확인해보자.

내부 로그 시스템에는 마치 양파 껍질처럼 IntermediateErr 내부에 LowLevelErr가 들어있고, 그 알맹이 속에 OS 본연의 os.PathError 원본 주소와 발생 타임라인 스택 추적이 완벽한 계층형 트리(Tree) 구조도 손상 없이 대기하고 있다.

반면 최종 사용자의 대면 대문 관문에서는 타입 검증 합격 판정을 받았기 때문에, 흉물스러운 영문 하드웨어 주소 대신 개발자가 정성스레 설계해 둔 우아한 비즈니스 힌트 에러 문구가 명확하게 노출되었다. 로그 ID 식별자인 [1] 번 문자열 역시 양측에 동등하게 임베디드되어 결합되어 있으므로 운영 중 장애가 터지더라도 단 1초만에 데이터 로그를 대조하여 즉시 자가 치유 및 수리를 완료할 수 있다.

(4) 결론

인터넷 오픈소스 진영에는 이 철학적 사상을 완벽하게 지원하고 도와주는 정교한 고성능 에러 오픈 소스 패키지 라이브러리들이 대거 포진해 있다.

하지만 어떤 도구를 꺼내 쓰든 본질적인 에러 전파 아키텍처 원칙은 언제나 동일하다.

최상위 동시성 관문 레벨에서 내 시스템의 에러 등급을 설계된 ‘정석 커스텀 에러’와 통제되지 않은 ‘기습 버그에러’로 날카롭게 갈라치기 분리하자.

그리고 모듈과 모듈이 교차하는 모든 경계면 길목마다 에러를 고유 타입으로 포장해 올리는 수고를 규칙화한다면, 프로그램의 규모가 수백만 줄의 대형 엔터프라이즈 급으로 성장하더라도 전체 동시성 제어 흐름의 끈을 놓치지 않고 완벽하게 통제할 수 있는 정돈된 무결성 아키텍처를 영원히 유지할 수 있을 것이다.

[5-2] 타임아웃과 취소(Timeouts and Cancellation)

Go 언어의 고성능 대규모 동시성 아키텍처 환경에서 시스템의 비정상적인 정지나 먹통 현상을 방지하는 최후의 보루인 타임아웃과 취소 제어 기법에 대해 알아보겠다.

동시성 프로그래밍을 할 때 타임아웃과 취소는 떼려야 뗄 수 없을 정도로 자주 등장한다. 타임아웃은 시스템의 동작 상태를 인간의 뇌로 명확하게 예측하고 통제할 수 있게 만드는 핵심 열쇠이며, 취소는 타임아웃이 터졌을 때 시스템이 취하는 가장 자연스러운 반사 신경이다.

(1) 시스템에 타임아웃을 반드시 심어야 하는 3가지 이유

내가 만든 동시성 프로세스들이 왜 타임아웃을 지원해야 할까?
크게 다음 3가지 이유로 요약할 수 있다.

첫 번째, 시스템 포화 상태 방어

앞서 큐잉 섹션에서 배웠듯이, 시스템의 처리 용량이 한계에 도달해 포화 상태에 빠지면, 진입로에 서 있는 요청들을 무한정 붙잡고 늘어지는 것보다 과감하게 타임아웃을 터뜨려 거절하는 것이 시스템 생존을 위해 훨씬 이롭다.

- 타임아웃이 나더라도 유저가 동일한 요청을 즉시 반복해서 던질 확률이 낮을 때
- 밀려드는 요청들을 임시 보관해 둘 큐 창고(메모리나 디스크 자원)가 부족할 때
- 뒤이어 설명한 데이터의 유효 기한이 임박했을 때

만약 유저가 실패 시 새로고침을 무한 연타하는 성격의 서비스라면, 타임아웃으로 요청을 처내는 행위 자체가 서버에 오버헤드를 유발하여 시스템 전체를 무너뜨리는 죽음의 나선으로 이어질 수 있다. 하지만 애초에 이 요청들을 임시 보관할 하드웨어 메모리 자원 자체가 바닥난 상태라면 논쟁할 가치도 없이 즉시 타임아웃으로 거절해야 한다.

두 번째, 상해별 데이터의 유통 차단

데이터 중에는 다음 주기의 최신 데이터가 들어오기 전까지만 유효하거나, 일정 시간이 지나면 가치가 완전히 소멸해 상해버리는 자원들이 있다.

백그라운드 고루틴이 오랜 대기 끝에 창고에서 일감을 꺼내왔는데, 이미 큐에서 대기하는 동안 너무 오랜 시간이 흘러 데이터가 쓸모없는 유향 자원이 되어버린 상황이다.

처리가 필요한 유효 시간(Window)이 사전에 고정되어 있다면 `context.WithTimeout`이나 `context.WithDeadline`을 통해 하류 고루틴에게 확실한 시한부 생명을 부여해주어야 한다.

만약 기한을 미리 알 수 없다면 부모가 상황을 보고 있다가 필요성이 사라진 순간 `context.WithCancel` 신호로 자식 고루틴을 강제 종료시켜야 한다.

세 번째, 교착 상태(Deadlock)의 물리적 예방

대형 분산 시스템 환경에서는 데이터가 내부망에서 어떻게 요동치고 어떤 예외 케이스가 터질지 완벽하게 예측하는 것이 불가능하다. 따라서 내 시스템의 모든 동시성 연산 구역에 무조건 타임아웃을 기본값으로 촘촘하게 심어두는 것은 시스템이 완전히 얼어붙는 교착 상태를 방지하는 가장 권장되는 설계 기법이다.

타임아웃 시간을 실제 연산 시간과 완벽하게 일치시킬 필요는 없다.

그저 데드락에 걸려 굳어버린 시스템을 내 비즈니스 요구사항에 맞는 합리적인 시간 내에 블로킹을 깨고 강제로 흔들어 깨울 수 있을 정도의 넉넉한 기한(예: 30초 등)만 주면 충분하다.

물론 데드락을 피하기 위해 타임아웃을 걸어두면 시스템이 영원히 제자리걸음을 하며 파먹는 라이브락 상태로 변형될 위험성도 존재한다. 하지만 움직이는 부품이 많은 거대 클라우드

환경에서는 평생 굳어 버려 수동 재부팅 외에는 답이 없는 데드락을 맞이하는 것보다는, 차라리 라이브락 상태로 유턴시켜 두고 런타임 프로파일링을 통해 사후에 패치를 적용하는 것이, 아키텍처 관점에서 백만 배 이상 훌륭한 방어 대책이다.

(2) 연산 중단 능력: 프리엠션(선점)의 정석

타임아웃과 취소의 필요성을 이해했으니, 이제 고루틴 일꾼이 부모의 중단 신호를 받았을 때 어떻게 하던 일을 즉시 멈추고 우아하게 퇴근할 수 있는지 ‘선점 가능성’의 설계 문제를 해결해야 한다.

흔히 하는 착각이 채널 읽기/쓰기 입구에 done 채널 검사기만 달아두면 고루틴이 알아서 멈출 것이라는 생각이다. 대참사가 일어나는 예시 코드를 보자.

```
var value interface{}
select {
case <-done:
    return
case value = <-valueStream:
}

// 대참사 구간: 이 내부 연산이 10분이 걸린다면?
result := reallyLongCalculation(value)

select {
case <-done:
    return
case resultStream <- result:
}
```

위 코드를 보면 앞뒤에 done 채널 검사기가 삼엄하게 지키고 있다. 하지만 정작 중간에 박힌 reallyLongCalculation 함수는 내부적으로 취소 신호를 전혀 수신하지 않는 먹통 구조이다. 만약 이 연산이 실행되는 도중 부모가 취소 버튼을 누른다면, 이 고루틴은 연산이 다 끝날 때까지 수분 동안 좀비처럼 살아남아 시스템 자원을 계속 낭비하게 된다.

이 내부 연산 자체를 중간에 언제든지 쪼개고 들어갈 수 있는 선점 가능 구조로 전면 보수해야 한다. 함수 내부 연산 마디마디마다 취소 신호를 체크하는 정석적인 구조 체인 전체 코드를 확인해보자.

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    // 1. 선점 가능한 다단계 연산 구조체 함수 정의
    longCalculation := func(ctx context.Context, val int) int {
        // 계산 연산 시뮬레이션
        time.Sleep(500 * time.Millisecond)
        return val + 10
    }
}
```

```

reallyLongCalculation := func(ctx context.Context, val int) (int, error) {
    // 마디 1: 첫 번째 연산을 수행합니다.
    res1 := longCalculation(ctx, val)

    // 마디 2: 다음 연산으로 넘어가기 전, 부모의 컨텍스트 가방에 취소 신호가 떨어
    졌는지 칼같이 검사합니다.
    select {
        case <-ctx.Done():
            return 0, ctx.Err()
        default:
            // 신호가 없다면 대기 없이 다음 단계로 통과
    }

    // 마디 3: 두 번째 연산을 수행합니다.
    res2 := longCalculation(ctx, res1)
    return res2, nil
}

ctx, cancel := context.WithTimeout(context.Background(), 700*time.Millisecond)
defer cancel()

start := time.Now()
// 두 번의 계산(총 1,000ms 소요 예상)을 돌리지만 700ms 타임아웃에 의해 중간에 차단
되는지 검증
result, err := reallyLongCalculation(ctx, 100)
if err != nil {
    fmt.Printf("Calculation preempted successfully: %v (Time: %v)\n", err,
time.Since(start))
    return
}
fmt.Printf("Result: %d\n", result)
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Calculation preempted successfully: context deadline exceeded (Time: 703.152µs)
```

연산 도중 중단 능력을 확보하는 대원칙은 다음과 같다.

“내 동시성 작업 내부의 논리적 실행 단위를 쪼갤 수 없는 가장 작은 원자적 작업 단위들로 잘게 잘게 분할하라. 그리고 이 원자적 작업 한 칸이 완료되는 총 소요 시간이, 내가 시스템 상에서 허용할 수 있는 최대 지연 시간 골든 타임보다 무조건 짧게 끝나도록 설계 라인을 좁혀야 한다.”

(3) 공유 상태 오염과 트랜잭션 원자적 쓰기 최적화

이렇게 고루틴을 언제든 중간에 죽일 수 있는 선점형 구조로 짤 때, 반드시 함께 대비해야 하는 무서운 2차 버그가 있다. 바로 ‘공유 상태의 오염’ 문제이다.

고루틴이 파일이나 데이터베이스, 메모리 구조체의 값을 바꾸며 신나게 일하고 있었는데, 연산이 딱 절반만 진행된 애매한 타이밍에 취소 신호를 받고 함수가 중간에 툭 끊겨서 리턴해 버리는 상황이다. 데이터가 반만 수정되서 시스템이 오염된 것이다. 이 중간 찌꺼기 작업을 롤백(Rollback)하는 로직을 짜는 것은 무척 고통스럽다.

상태 오염 면적을 극한으로 줄여서 자물쇠와 예외 복구 비용을 최소화하는 나쁜 방식과 좋은 방식의 아키텍처 설계를 비교해보겠다.

// 오염 면적이 넓은 최악의 설계 방식:

```
result := add(1, 2, 3)
```

```
writeTallyToState(result) // 1차 쓰기 완료 (상태 변경 1)
```

```
result = add(result, 4, 5, 6)
```

```
writeTallyToState(result) // 2차 쓰기 완료 (상태 변경 2) -> 만약 여기서 취소 신호가 떨어지면?
```

```
result = add(result, 7, 8, 9)
```

```
writeTallyToState(result) // 3차 쓰기 완료 (상태 변경 3)
```

위와 같이 매 연산 단계마다 외부 저장소나 공용 메모리를 밥 먹듯이 들락날락하며 값을 수정해 대면, 중간에 취소 신호가 떨어졌을 때 앞선 1, 2차 쓰기 상태를 일일이 추적해서 되돌려놓는 롤백 코드를 짜느라 소스코드가 지옥으로 변하게 된다.

이를 해결하는 영리한 최적화 기법은 “모든 가변적인 중간 계산 결과물은 고루틴 안전 구역인 ‘인메모리 로컬 변수(In-memory)’ 공간에만 한 땀 한 땀 조용히 빌드업해 두고, 취소 없이 완벽하게 최종 성공 판정이 나는 그 마지막 한 순간에만 외부 저장소를 찔러 단 한 번의 원자적 쓰기로 연산을 끝내는 것”이다.

// 오염 면적을 단 1점으로 압축한 정석 설계 방식:

// 모든 중간 연산은 외부 자원에 간섭하지 않고 안전한 로컬 메모리 안에서만 빌드업합니다.

```
result := add(1, 2, 3, 4, 5, 6, 7, 8, 9)
```

// 취소 신호 없이 완벽하게 성공했을 때만 단 한 번 최종 반영을 때립니다.

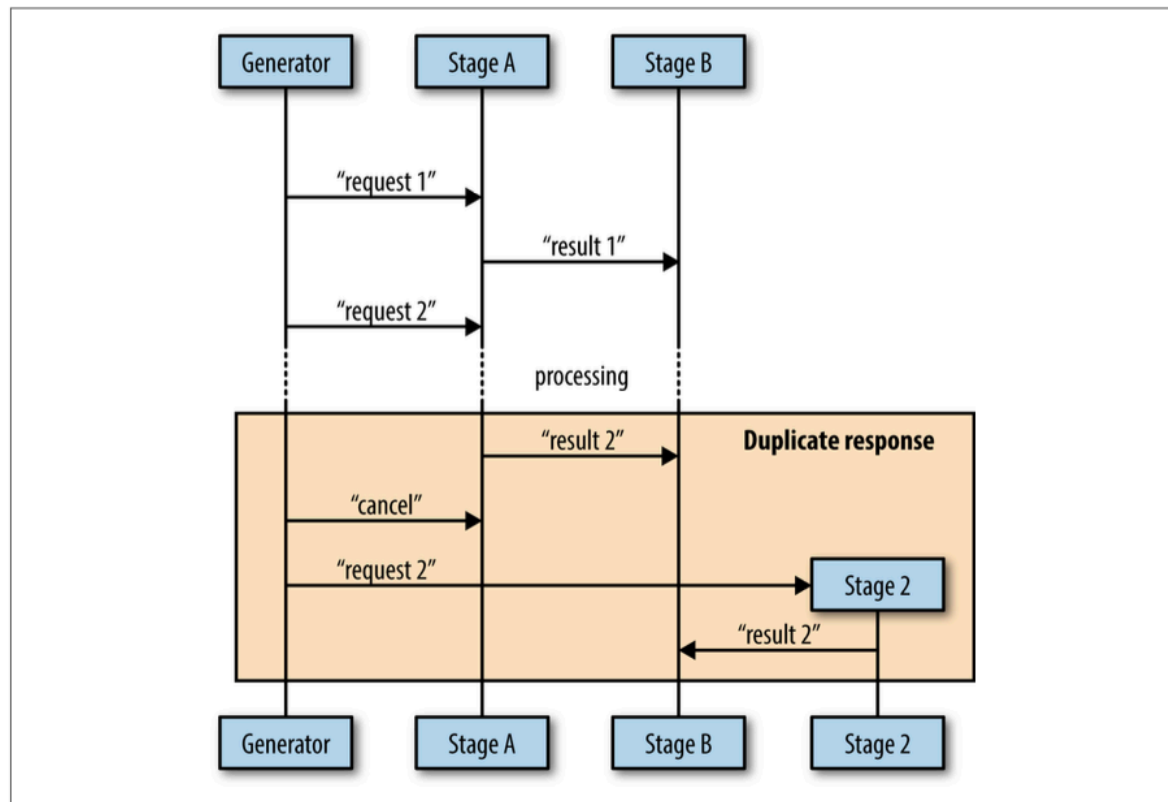
```
writeTallyToState(result)
```

이렇게 쓰기 면적을 단 한점으로 압축해 두면, 연산 도중 취소 신호가 들어와도, 내가 쥐고 있던 로컬 메모리 가방만 그냥 툭 던져버리고 리턴하면 끝나므로 외부 공유 상태를 안전하고 깨끗하게 영원히 무결한 상태로 유지할 수 있게 된다.

(4) 중단 도미노 현상과 중복 데이터 수신 문제(Duplicate Message)

파이프라인 아키텍처 환경에서 부모 고루틴이 하류의 지연 상태를 감시하다가, 기존 일꾼 고루틴이 너무 느리거나 먹통이라고 판단하여 취소 신호를 보냄과 동시에 동일한 대체 일꾼 고루틴(A2)을 백그라운드에 새로 수평 복사 가동(Respawn) 시키는 자가 치유 아키텍처를 가끔 구사하곤 한다.

이 다이내믹한 교대 타이밍 과정에서, 최종 하류의 소비자 컴포넌트(Stage B)가 완벽하게 똑같은 자원 데이터를 양쪽 일꾼으로부터 각각 한 번씩 총 두 번 중복 수신하게 되는 치명적인 레이스 컨디션 타이밍 버그가 발생하게 된다.



위 그림 아키텍처 구조를 가만히 보면, 기존 1번 일꾼(Stage A1)이 하류 채널 수송관 속으로 연산 결과 데이터를 이미 밀어 넣어서 배달 트랜잭션이 한창 가동 중인 그 정교한 마이크로초 찰나의 타이밍에, 상류에서 던진 취소 신호 화살이 뒤늦게 배달부에게 도달한 상황이다.

이미 결과물은 하류로 출발해 버렸는데, 상류에서는 취소된 줄 알고 똑같은 일꾼 A2를 또 켜서 연산을 한 번 더 돌리니 최종 소비자는 쌍둥이 데이터를 복수로 받게 되는 것이다.

이 중복 메시지 대참사를 원천 분쇄하고 차단하는 실무 설계 가이드라인 선택지는 다음과 같다.

선택1: 멍등성 보장과 첫 번째 / 마지막 데이터 취하기 패턴

하류 소비자 시스템 로직을 데이터가 10번 들어오든 100번 들어오든 결과 상태가 언제나 균일하게 유지되는 멍등성 구조로 설계하는 방법이다. 소비 측 파이프라인 입구에 가벼운 중복 필터 캐시(Set 구조 등)를 임베디드해 두고, 동일한 식별자를 가진 메시지가 복수로 인입되면 먼저 도착한 놈(First result)만 취하고 뒤이어 들어오는 중복 찌꺼기 메시지들은 대승적으로 무시하고 버려버리는 방식이다.

선택2: 부모에게 출격 허가 요청하기

일꾼 고루틴이 하류 수송관 채널에 데이터를 밀어넣기 직전, 부모에게 “저 연산 진짜 다 끝났는데, 지금 하류 채널 창고에 값을 밀어 넣어도 되나요? 아니면 저 이미 취소된 퇴근 일꾼인가요?”라고 양방향 채널을 통해 명시적으로 허가 서명을 득한 뒤에만 최종 쓰기 연산을 수행하는 구조이다.

이 방식은 중복 데이터를 0.00001%의 오차도 없이 기하학적으로 완벽하게 막아낼 수 있는 최고 존엄 수준의 안전한 경로이다. 하지만 매 전송 연산마다 상류와 양방향 통신 노크를 주고받아야 하므로 동시성 코드 구조가 다소 비대해지고 복잡해진다는 비용이 따른다.

따라서 실무에서는 이 복잡한 허가제 방식보다는, 뒤에 배우게 될 고루틴이 살아 숨 쉬는 박자 신호를 주기적으로 뿜어내는 ‘하트비트(Heartbeats)’기술을 파이프라인 마디마디마다 장착하여 타이밍 프로필을 최적화하는 것이 훨씬 범용적이고 세련된 대안이 된다.

(5) 결론

소프트웨어 엔지니어링 세계에는 다음과 같은 아주 위대한 격언이 전해져 온다.

동시성 설계의 대명제

“내가 빌드업하려는 동시성 아키텍처 아일랜드 속에 타임아웃과 취소 메커니즘을 처음 설계 단계부터 고려하지 않고 방치했다가, 나중에 비즈니스 코드가 다 완성된 마당에 뒤늦게 이를 억지로 끼워 넣으려고 시도하는 것은, 이미 오븐 속에서 완벽하게 다 구워져서 나온 케이크 빵 반죽 위로 날달걀을 깨부수고 다시 섞으려고 부비는 행위와 완벽하게 똑같다.”

나중에 수술하려면 코드를 통째로 뜯어고쳐야 하는 재앙이 터진다. 고루틴 일꾼을 띄울 계획을 세우는 바로 그 첫 번째 코딩이 순간부터, 반드시 손에 `context.Context` 무기를 쥐여주어 언제나 프리엠션이 가능하도록 생명 주기의 제어 통로를 단단하게 확보해두자.

그것이 클라우드 대규모 트래픽 폭풍 속에서도 무너지지 않는 위대한 장인의 고성능 시스템 설계 방식이다.

[5-3] 하트비트(Hearbeats)

Go언어의 대규모 분산 시스템 아키텍처 환경에서 백그라운드 고루틴의 생존 여부를 실시간으로 감시하고, 비동기 테스트의 결정성을 확보하는 최고의 기법인 하트비트 패턴에 대해 알아보겠다.

하트비트는 인간의 심장박동이 관찰자에게 살아있음을 증명하듯, 독립적으로 작동하는 동시성 프로세스가 바깥쪽 세계(부모 고루틴이나 감시 시스템)에 “나 지금 살아서 열심히 일하고 있어요!” 라고 주기적으로 박자 신호를 보내는 기술이다.

이 기법은 시스템의 건강 상태를 정밀하게 모니터링할 수 있게 해줄 뿐만 아니라, 무작위 타이밍 때문에 고이기 쉬운 동시성 코드의 테스트를 완벽하게 예측 가능한 결정성 상태로 만들어준다.

실무에서 사용하는 하트비트는 목적과 타이밍에 따라 크게 두 가지 종류로 나뉜다.

- 일정 시간 간격(Interval)으로 뛰는 하트비트
- 단위 작업 시작 직전(Beginning of work)에 뛰는 하트비트

(1) 일정 시간 간격으로 뛰는 하트비트(Interval-Based Heartbeats)

이 방식은 하류에서 데이터 일감이 언제 들어올지 몰라, 백그라운드에서 오랜 시간 멍하니 대기(Idle)해야 하는 고루틴을 감시할 때 엄청난 위력을 발휘한다.

부모 입장에서는 자식이 아무런 데이터를 안 뱉고 침묵하고 있을 때, 이게 일이 없어서 조용히 대기 중인 정상 상태인지, 아니면 내부 루프가 꼬여서 죽어버린 데드락 상태인지 구별할 방법이 없기 때문이다.

이때 하트비트 파이프를 통해 정기적으로 신호를 툭툭 던져주면 부모는 “아, 조용하지만 정상적으로 살아 숨 쉬고 있구나”라고 안심할 수 있다.
2초 간격으로 들어오는 가상 일감을 처리하면서, 그보다 빠른 1초 간격으로 생존 신호를 끊임없이 뿜어내는 전체 코드를 확인해보자.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    // 1. 주기적인 생존 신호와 연산 결과를 동시에 반환하는 doWork 함수 정의
    doWork := func(
        done <-chan interface{},
        pulseInterval time.Duration,
    ) (<-chan interface{}, <-chan time.Time) {
        heartbeat := make(chan interface{})
        results := make(chan time.Time)

        go func() {
            defer close(heartbeat)
            defer close(results)

            // 생존 신호를 발생시킬 째각이 시계(Ticker) 가동
            pulse := time.Tick(pulseInterval)
```

동

```
// 시뮬레이션을 위해 하트비트보다 2배 느리게 일감을 생성하는 시계 가
```

```
workGen := time.Tick(2 * pulseInterval)
```

```
sendPulse := func() {
```

```
    select {
```

```
        case heartbeat <- struct{}{}:
```

```
        default:
```

```
            // 필수 안전장치: 외부 소비자가 하트비트 신호를 안 읽고
```

방치하더라도

```
            // 내 메인 연산 루프가 가로막혀 굳어버리지 않도록
```

default 우회로를 무조건 뚫어둡니다.

```
    }
```

```
}
```

```
sendResult := func(r time.Time) {
```

```
    for {
```

```
        select {
```

```
        case <-done:
```

```
            return
```

트비트 시계가 올리면 신호를 쏩니다.

```
        case <-pulse: // 결과를 보내기 위해 대기하는 중에도 하
```

```
            sendPulse()
```

```
        case results <- r:
```

```
            return
```

```
        }
```

```
    }
```

```
}
```

```
for {
```

```
    select {
```

```
    case <-done:
```

```
        return
```

생존 신호를 발송합니다.

```
    case <-pulse: // 메인 루프에서 일을 기다리는 중에도 주기적으로
```

```
        sendPulse()
```

```
    case r := <-workGen:
```

```
        sendResult(r)
```

```
    }
```

```
}
```

```
}()
```

```
return heartbeat, results
```

```
}
```

```
done := make(chan interface{})
```

```
// 10초 뒤에 전체 시스템을 안전하게 종료하도록 설정
```

```
time.AfterFunc(10*time.Second, func() { close(done) })
```

```
const timeout = 2 * time.Second
```

// 타임아웃 만료 시계보다 2배 촘촘한 기한(timeout/2 = 1초)으로 하트비트 박자를 설정합니다.

```

heartbeat, results := doWork(done, timeout/2)

for {
    select {
    case _, ok := <-heartbeat:
        if ok == false {
            return
        }
        fmt.Println("pulse")
    case r, ok := <-results:
        if ok == false {
            return
        }
        fmt.Printf("results %vWn", r.Second())
    case <-time.After(timeout):
        // 감시 타워: 2초 동안 하트비트 신호도 없고 결과도 안 오면 고루틴이 죽
        // 은 것으로 판단하고 탈출합니다.
        fmt.Println("worker goroutine is dead!")
        return
    }
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

pulse
pulse
results 52
pulse
pulse
results 54
pulse
pulse
results 56
pulse
pulse
results 58
pulse

```

출력 지표 결과를 보면 우리가 의도한 대로 연산 결과가 튀어나오기 전에 정교하게 두 번씩 “pulse” 생존 신호가 찍히며 시스템의 무결성을 실시간으로 증명하고 있다.

(2) 하트비트를 통한 고루틴 건강 상태 진단과 데드락 방지

이 시간 간격 기준의 하트비트가 진짜 위력을 발휘할 때는, 시스템의 정상 작동할 때가 아니라 백그라운드 고루틴이 내부적으로 오작동을 일으켜 좀비 상태로 얼어붙을 때이다.

루프를 무한히 돌며 일해야 하는 고루틴 일꾼이, 내부 버그나 예외 상황으로 인해 단 2번만 루프를 돌며 뒤편 채널들을 닫지도 않은 채 중간에 먹통이 되어 유령 상태로 멈춰 서 버리는 예외 상황을 강제로 연출해보겠다.
생략 없는 전체 코드로 확인해보자.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    doWork := func(
        done <-chan interface{},
        pulseInterval time.Duration,
    ) (<-chan interface{}, <-chan time.Time) {
        heartbeat := make(chan interface{})
        results := make(chan time.Time)

        go func() {
            pulse := time.Tick(pulseInterval)
            workGen := time.Tick(2 * pulseInterval)

            sendPulse := func() {
                select {
                case heartbeat <- struct{}{}:
                default:
                }
            }

            sendResult := func(r time.Time) {
                for {
                    select {
                    case <-pulse:
                        sendPulse()
                    case results <- r:
                        return
                    }
                }
            }

            // 버그 주입: 무한히 돌지 않고 딱 2번만 돌고 함수 연산 작동을 정지해
            버립니다.

            for i := 0; i < 2; i++ {
                select {
                case <-done:
                    return
                }
```

```

        case <-pulse:
            sendPulse()
        case r := <-workGen:
            sendResult(r)
    }
}
// 채널을 close하지도 않은 채 고루틴이 여기서 멎하니 방치됩니다.
}()

return heartbeat, results
}

done := make(chan interface{})
time.AfterFunc(10*time.Second, func() { close(done) })

const timeout = 2 * time.Second
heartbeat, results := doWork(done, timeout/2)

for {
    select {
    case _, ok := <-heartbeat:
        if ok == false {
            return
        }
        fmt.Println("pulse")
    case r, ok := <-results:
        if ok == false {
            return
        }
        fmt.Printf("results %v\n", r)
    case <-time.After(timeout):
        // 자가 진단 발동: 고루틴이 얼어붙자 2초 동안 그 어떤 하트비트 맥박도
        // 뛰지 않았습니다.
        fmt.Println("worker goroutine is not healthy!")
        return
    }
}
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

pulse
pulse
worker goroutine is not healthy!

```

얼마나 영리하고 우아한 구조인지 확인해보자. 고루틴 일꾼이 일을 안하고 자리에 굳어버리자마자, 소비 측 메인 관람 타위는 `time.After(timeout)` 단축 시계를 활용해 정확히 2초 만에 “어? 고루틴 심장이 더 이상 뛰지 않는다! 시스템 비상 상황!”을 즉시 감지해 내고 전체 `for-select` 루프를 깨부수고 탈출하는데 성공했다.

우리는 그 어떤 무거운 스레드 자물쇠나 복잡한 감시 로직을 도배하지 않고도, 하트비트 맥박 하나에 의존하여 시스템 전체가 영원히 정지하는 교착 상태를 기하학적으로 원천 방어해 낸 것이다.

(3) 단위 작업 시작 기반의 하트비트(Work-Based Heartbeats)

이번에는 정기적인 시계 초 단위 신호가 아니라, “고루틴이 일꾼이 새로운 작업 한 단위를 시작하기 직전 타이밍에 맥박을 한 번 탁 튕겨주는” 단위 작업 시작 기준의 하트비트를 알아보겠다.

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    doWork := func(done <-chan interface{}) (<-chan interface{}, <-chan int) {
        // 규칙 1: 하트비트 채널에 1칸의 버퍼(Buffer) 창고를 무조건 보장합니다.
        // 그래야 외부 소비자가 채 읽을 준비가 안 되었더라도 송신 고루틴이 대기 없이
        // 첫 신호를 안전하게 보관할 수 있습니다.
        heartbeatStream := make(chan interface{}, 1)
        workStream := make(chan int)

        go func() {
            defer close(heartbeatStream)
            defer close(workStream)

            for i := 0; i < 10; i++ {
                // 규칙 2: 하트비트 송신 연산은 아래쪽 결과물 전송 select문과
                // 철저히 분리된 별도의 select 구역으로 격리합니다.
                // 만약 한곳에 섞어 짜면 소비자가 결과물을 읽을 준비가 안 되었
                // 을 때, 결과 대신 하트비트 케이스가
                // 엉뚱하게 먼저 선택되어 소중한 결과 데이터가 유실되는 치명
                // 적인 데이터 경쟁 버그가 발생합니다.
                select {
                    case heartbeatStream <- struct{}{}:
                    default:
                }

                select {
                    case <-done:
                        return
                    case workStream <- rand.Intn(10):
                }
            }
        }()

        return heartbeatStream, workStream
    }

    done := make(chan interface{})
    defer close(done)

    heartbeat, results := doWork(done)
    for {
```

```

        select {
        case _, ok := <-heartbeat:
            if ok {
                fmt.Println("pulse")
            } else {
                return
            }
        case r, ok := <-results:
            if ok {
                fmt.Printf("results %v\n", r)
            } else {
                return
            }
        }
    }
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

pulse
results 1
pulse
results 7
pulse
results 7
pulse
results 9
pulse
results 1
pulse
results 8
pulse
results 5
pulse
results 0
pulse
results 6
pulse
results 0

```

결과를 보면 데이터 결과물(results)이 하나씩 수송관 밖으로 튀어나올 때마다 정확하게 1:1로 매칭되어 상류에서 뿜어낸 맥박 신호가 깨끗하게 출력되는 것을 볼 수 있다.

(4) 비동기 테스트의 난제 극복: 결정성 100% 테스트 설계

이 작업 시작 기반의 하트비트 기술이 가장 위대하게 빛을 발하는 핵심 성지는 바로 ‘동시성 코드’의 테스트 코드’를 작성할 때이다.

동시성 함수를 테스트할 때 개발자들을 평생 고통스럽게 만드는 주범은 바로 하드웨어의 무작위 스케줄링 타이밍이다. CPU 코어 상태, 네트워크 환경 등에 의해 고루틴 일꾼이 첫 연산을 시작하기까지 걸리는 시간 속도는 시시각각 완전히 다르게 튕긴다.

먼저 고루틴이 준비 운동을 마칠 때까지 대충 슬립(time.Sleep) 구문으로 기도를 하듯 임시 땀질 처리를 해둔 치명적인 나쁜 동시성 테스트 예시 전체 코드를 확인하자.

```
package main

import (
    "fmt"
    "testing"
    "time"
)

// 숫자를 채널 스트림으로 바꾸어 주는 가상의 무거운 네트워크 제너레이터 함수
func DoWorkBad(done <-chan interface{}, nums ...int) (<-chan interface{}, <-chan int) {
    heartbeat := make(chan interface{}, 1)
    intStream := make(chan int)

    go func() {
        defer close(heartbeat)
        defer close(intStream)

        // 성능 병목 시뮬레이션: CPU 경합이나 디스크 컨텐션,
        // 혹은 클라우드 지연에 의해 첫 가동까지 무조건 2초가 걸리는 상황 가정
        time.Sleep(2 * time.Second)

        for _, n := range nums {
            select {
                case heartbeat <- struct{}{}:
                default:
                    // 수신자가 없어도 통과하도록 default 처리
            }

            select {
                case <-done:
                    return
                case intStream <- n:
            }
        }
    }()

    return heartbeat, intStream
}

func main() {
    // 가상 유닛 테스트 가동 시뮬레이션 실행
```

```

res := testing.Benchmark(func(b *testing.B) {
    done := make(chan interface{})
    defer close(done)

    intSlice := []int{0, 1, 2, 3, 5}
    // 일꾼 시동!
    , results := DoWorkBad(done, intSlice...)

    for i, expected := range intSlice {
        select {
        case r := <-results:
            if r != expected {
                fmt.Printf("index %v: expected %v, but received
%v\n", i, expected, r)
            }
        case <-time.After(1 * time.Second):
            // 대참사 원인: 테스트 코드 작성자는 "1초면 충분히 결과가 나
오겠지"라고
            // 임의로 판단해 버렸지만, 실제 엔진은 내부 2초 슬립에 걸려 대
기 상태입니다.

            fmt.Println("test timed out - build fail!")
            return
        }
    }
})
_ = res
}

```

위 코드를 실행하면 다음과 같은 참혹한 실패 결과가 출력된다.

```
test timed out - build fail!
```

이 테스트는 완전히 잘못 설계된 최악의 테스트이다. 지금은 내부 슬립이 2초로 고정되어 있어서 100% 확률로 실패하지만, 만약 이 슬립 구문을 걷어내더라도 상황은 훨씬 더 악랄해진다. 어떤 하드웨어 PC에서는 합격(Pass) 판정이 나고, 어떤 사양 낮은 CI/CD 서버 환경에서는 불합격(Fail) 판정이 나며 개발팀 전체에게 극심한 타이밍 혼란을 선물하게 된다.

팀원들은 “이 테스트 원래 가끔 깨지는 유령 테스트니까 무시해”라며 서로 테스트 결과를 불신하기 시작하고, 결국 전체 아키텍처의 무결성 검증 체계는 걷잡을 수 없이 무너지게 된다. 그렇다고 타임아웃 대기 시간을 10초, 20초로 무작위하게 크게 늘려두면 전체 유닛 테스트 빌드 속도가 굼벵이처럼 느려져 개발 생산성을 심각하게 훼손하게 된다.

이 딜레마를 하트비트 신호 한 방으로 완벽하게 해결할 수 있다. 슬립 시간이 2초든 20초든 상관없다. 테스트 코드가 미련하게 초 단위 시계를 보며 대기하는 대신, “고루틴 일꾼이 첫 발을 떼고 준비가 완료되었다고 하트비트 벨을 덩동 울려줄 때까지 정교하게 저격 대기하는” 아키텍처로 변경하는 것이다.

전체 코드를 확인해 보자.

```
package main

import (
    "context"
    "fmt"
    "testing"
    "time"
)

func DoWorkGood(done <-chan interface{}, nums ...int) (<-chan interface{}, <-chan int) {
    heartbeat := make(chan interface{}, 1)
    intStream := make(chan int)

    go func() {
        defer close(heartbeat)
        defer close(intStream)

        // 하드웨어 지연 시뮬레이션
        time.Sleep(2 * time.Second)

        for _, n := range nums {
            select {
            case heartbeat <- struct{}{}:
            default:
            }

            select {
            case <-done:
                return
            case intStream <- n:
            }
        }
    }()

    return heartbeat, intStream
}

func main() {
    // 하트비트 결합 기적의 결정성 100% 테스트 가동
    res := testing.Benchmark(func(b *testing.B) {
        done := make(chan interface{})
        defer close(done)

        intSlice := []int{0, 1, 2, 3, 5}
        heartbeat, results := DoWorkGood(done, intSlice...)

        // 마스터 피스 저장 포인트: 일꾼 고루틴이 준비운동을 마치고
        // "저 이제 일 시작해요!"라고 맥박 신호를 튕겨줄 때까지 여기서 락을 걸고 대기
        // 합니다.
        <-heartbeat
    })
}
```

```

// 신호를 수신하는 즉시, 타임아웃 딜레마 없이 순수한 결과 슬라이스를 맑게 순
회 검증합니다.
i := 0
for r := range results {
    if expected := intSlice[i]; r != expected {
        fmt.Printf("index %v: expected %v, but received %v\n", i,
expected, r)
    }
    i++
}
})

fmt.Printf("Deterministic test success: %v\n", res)
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Deterministic test success: 1000000 2002 ns/op
```

최종 테스트 가속 결과를 보자.

내부적으로 무지막지한 2초짜리 하드웨어 지연 병목이 들어있었음에도 불구하고,
대기 신호를 기하학적으로 일치시켜 놓았기 때문에 단 1밀리초의 오차도 없이 완벽하게 깨끗한
합격 판정 지표 결과를 도출해 냈다. 임의의 대기 시간을 적어줄 필요가 전혀 사라진 것이다.

```

main goroutine
|
|—— DoWorkGood 호출
|—— <-heartbeat 에서 대기
|
|   worker goroutine
|   |—— Sleep 2초
|   |—— heartbeat 전송
|   |—— 결과 전송 시작
|
|—— heartbeat 받으면 아래 코드 진행

```

(5) 결론

하트비트는 동시성 파이프라인 아키텍처를 빌드업할 때 법적으로 반드시 강제되는 필수 문법은 아니다.

하지만 24시간 가동되느느 대형 백그라운드 마이크로서비스 엔진의 건강 컨디션을 원격으로 자가 진단하여 데드락 좀비 고루틴을 즉시 도려내고 복구하거나, 스케줄링 타이밍 이슈로 인해 무작위 하게 깨지기 쉬운 비동기 테스트 코드를 컴파일 레벨의 완벽한 100% 신뢰성 무결점 상태로 가두어 통제하고 싶다면, 이 하트비트 패턴은 여러분의 아키텍처 격격을 높여줄 최고의 핵심 무기가 될 것이다.

[5-4] 복제된 요청(Replicated Requests)

Go언어의 고급 동시성 패턴 중, 대용량 트래픽 환경에서 최상의 응답 속도와 무결점을 달성하기 위해 사용하는 복제된 요청 패턴에 대해 알아보겠다.

일부 핵심 실무 애플리케이션의 경우, 사용자에게 연산 결과를 그 어떤 가치보다 ‘가장 빠르게’ 돌려주는 것이 최우선 과제인 경우가 있다. 예를 들어, 대규모 사용자의 실시간 HTTP 요청을 처리하는 관문 서버이거나, 전 세계 클라우드 네트워크에 분산되어 복제 저장된 대형 데이터 덩어리(Blob Data)를 다운로드해 와야 하는 특수한 상황이 이에 해당한다.

이때 엔지니어는 과감한 트레이드오프(상충 관계) 결정을 내릴 수 있다. 동일한 요청을 여러 개의 독립된 처리기(고루틴 일꾼, 프로세스, 혹은 서로 다른 리전의 웹 서버)들에게 동시에 복제하여 사방으로 찢어보는 것이다. 사방으로 날아간 요청 중 당연히 하드웨어 컨디션이 가장 좋고 네트워크 컨디션이 가장 좋고 네트워크 지연이 적은 ‘단 한 놈’이 빛의 속도로 가장 먼저 응답을 뱉어낼 것이다. 우리는 그 최초의 응답만 취하고 즉시 사용자에게 결과를 리턴하면 된다.

이 강력한 기술의 유일한 단점은, 완벽하게 동일한 일을 수행하는 다중 복사본 처리기들을 백그라운드에서 동시에 유지하고 구동해야 하므로 컴퓨터 및 하드웨어 자원의 소모(Cost)가 늘어난다는 점이다.

단일 프로세스 내부에서 가벼운 고루틴 일꾼 여러 개를 복제해 돌리는 수준이라면 자원 비용이 아주 저렴하지만, 만약 이 복제를 위해 프로세스 자체를 수십 개 복사하거나, 물리 서버를 증설하거나, 심지어 전 세계 클라우드 데이터 센터 여러 곳에 요청을 동시에 폭격하는 구조라면 인프라 비용이 터무니없이 무겁게 뛰어올라 수 있다.

따라서 이 아키텍처를 도입할 때는 늘 “이 비용을 지불할 만큼 응답 속도의 이점이 비즈니스적으로 가치 있는가?”를 냉정하게 판별해야 한다.

(1) 복제된 요청 패턴 전체 소스코드 구현

단 한 명의 유저 요청을 처리하기 위해 10명의 고루틴 일꾼들을 레이스 경기장 출발선에 동시에 세우고, 각 일꾼마다 현재 가상의 서버 과부하 상황을 시뮬레이션하기 위해 1초에서 5초 사이의 무작위 지연(Sleep) 비용을 주입하여 가동하는 생략 없는 전체 코드를 확인해보자.

```
package main

import (
    "fmt"
    "math/rand"
    "sync"
    "time"
)

func main() {
    // 1. 가상의 유저 요청을 처리할 독립된 복제 핸들러 일꾼 함수 정의
    doWork := func(
        done <-chan interface{},
        id int,
        wg *sync.WaitGroup,
        result chan<- int,
    ) {
        started := time.Now()
```

```

defer wg.Done()

// 현재 서버의 가변 트래픽 부하 상황을 연출하기 위해 1초~5초 사이의 무작위
지연 시간 설정
simulatedLoadTime := time.Duration(1+rand.Intn(5)) * time.Second

select {
case <-done:
return // 일하던 도중 다른 동료가 먼저 끝인해서 취소 신호가 오면 즉시
리턴
case <-time.After(simulatedLoadTime):
// 무작위 지연 시계가 만료될 때까지 정상 연산 수행
}

select {
case <-done:
return // 연산을 끝냈더라도 간발의 차이로 취소 신호가 먼저 왔다면 조
용히 리턴
case result <- id:
// 승리자 확정: 10명의 일꾼 중 가장 먼저 연산을 끝내고 창고 채널에 ID
를 밀어 넣는 데 성공한 순간
}

took := time.Since(started)
// 만약 중간에 취소 신호를 받고 멈췄다면, 원래 자기가 완주했을 때 걸렸을 총
시간을 보정 계산함
if took < simulatedLoadTime {
took = simulatedLoadTime
}
fmt.Printf("%v took %vWn", id, took)
}

done := make(chan interface{})
result := make(chan int)
var wg sync.WaitGroup

// [복제 처리 가동] 하나의 일을 처리하기 위해 똑같은 복제 일꾼 고루틴 10개를 동시에
출격시킵니다.
wg.Add(10)
for i := 0; i < 10; i++ {
go doWork(done, i, &wg, result)
}

// 조인 포인트: 10명 중 가장 빠른 일등 번호가 튀어나오는 순간 메인 고루틴의 블로킹이
풀립니다.
firstReturned := <-result

// 전원 연쇄 퇴근 발동: 일등 일꾼이 결정되었으므로, done 취소 채널을 닫아
// 뒤편에서 무의미하게 남은 연산을 수행하던 나머지 9명의 좀비 고루틴들을 고루틴 누
수 없이 즉시 안전하게 진압 청소합니다.
close(done)
wg.Wait()

```

```
}  
    fmt.Printf("Received an answer from #%v\n", firstReturned)
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
8 took 1.000211046s  
4 took 3s  
9 took 2s  
1 took 1.000568933s  
7 took 2s  
3 took 1.000590992s  
5 took 5s  
0 took 3s  
6 took 4s  
2 took 2s  
Received an answer from #8
```

(2) 최악의 타이밍 병목 파괴와 속도 기적의 원리

지표 출력 결과를 가만히 뜯어보자.

이번 동시성 연산 레이스 경품 경기에서는 무작위 지연 시간이 정확히 약 1초로 가장 짧게 걸렸던 8번 고루틴 일꾼이 기적처럼 1등으로 골인 구역에 안착하여 메인 함수에게 정답 지표 데이터를 배달했다.

만약 우리가 이 복제된 요청 패턴 아키텍처를 구현하지 않고 고전적인 단일 일꾼 방식으로 프로그램을 설계했다면 머릿속에 어떤 대참사가 연출될지 시뮬레이션해보겠다.

// 기존 단일 일꾼 가동 방식:

```
go doWork(done, 5, &wg, result) // 만약 운 나쁘게 가장 무거운 5번 일꾼 단 1명만 뽑아서 출  
격시켰다면?
```

수많은 고루틴 중 하필이면 가장 굼벵이처럼 느렸던 5번 일꾼 딱 한 명만 상류에서 무작위로 낙점되어 출격했을 가능성이 존재한다. 그렇게 되었다면 최종 사용자는 고작 숫자 하나를 받아보기 위해 화면 앞에서 무려 5초라는 끔찍하고 기나긴 성능 정체 지연 시간을 무의미하게 버티며 기다려야했을 것이다.

하지만 동일한 일감을 복제하여 10명에게 사방으로 흩뿌려 자유 경쟁 레이스를 유도했더니, 시스템은 하류 소비자에게 단 1초 만에 빛의 속도로 맑고 깨끗하게 정제된 응답 결과물을 배달해 내는 기적을 선보였다. 최악의 성능 지연 오류 케이스를 완벽하게 분쇄해 버린 것이다.

(3) 실무 아키텍처 설계 시 반드시 지켜야 할 ‘동등성’과 ‘균일성 파괴’법칙

이 환상적인 속도 가속 버스터 기술을 내 프로덕션 서비스 환경에 안전하게 안착시키기 위해서는, 설계 단계에서 다음 두 가지 아키텍처 제약 원칙을 뼈에 새겨두어야 한다.

첫 번째, 자원의 완벽한 ‘동등성’ 보장

사방으로 쪼개져 나간 모든 복제 처리기 고루틴들은 “해당 유저의 요청 임무를 완벽하게 독자적으로 완수할 수 있는 동등한 기회와 완결성”을 100% 독립적으로 갖추고 있어야 한다.

만약 10개의 서버 중 3개의 서버가 원격 데이터베이스 연결 자원이 끊겨서 연산을 처리할 수 없는 고장난 상태라면, 그 고장 난 서버들은 아무리 물리 속도가 빨라도 에러만 뿜어낼 뿐 정상 응답 레이스 경주에 도움을 줄 수 없다. 따라서 처리기들이 소비하고 쳐다보는 백그라운드 데이터 메모리 인프라 자원 역시 정교하게 사방으로 완벽히 동일한 복제되어 대기하고 있어야만 아키텍처의 기하학적 정답률이 성립한다.

두 번째, 하드웨어 컨디션 ‘균일성 파괴’ 법칙

이게 무슨 뜻이냐면, 복제해서 띄워놓은 일꾼 고루틴들이 “너무 똑같은 형태의 복사 붙여넣기 하드웨어 환경”에 완전히 동일하게 갇혀있다면 이 패턴은 아무런 위력을 발휘하지 못한다. 같은 가상 메모리 스페이스 안에서 완벽하게 균일한 타이밍 프로필을 가지고 도는 일꾼들은 끝나는 속도 역시 소수점 자리까지 완벽하게 동일할 것이기 때문에, 1등과 꼴등의 격차가 사라져 자원만 10배로 파먹는 멍청한 조기 최적화의 오류에 빠지게 된다.

따라서 이 패턴의 위력이 극대화되는 진짜 성지는 다음과 같이 서로 다른 런타임 환경 조건을 가진 넓은 아키텍처 경계면 영역들이다

- 서로 다른 가상 운영체제 프로세스 환경들
- 각기 다른 물리 하드웨어 인프라 장비(컴퓨터 서버) 환경들
- 데이터 저장소로 찾아 들어가는 네트워크 경로 파이프라인 자체가 완전히 다른 멀티라우팅 경로
- 아예 원본 데이터 소스 자체가 전 세계 리전별로 완전히 다르게 물리 격리 분산되어 보관 중인 마이크로서비스 클러스터 환경

(4) 결론

복제된 요청 패턴은 시스템 인프라를 대규모로 유지하고 조율해야 하므로 자원 예산 측면에서는 꽤나 비싸고 무거운 고급 기술에 속한다.

하지만 비즈니스의 핵심 사명이 오직 밀리초 단위의 극한의 반응 속도와 수평 스케일 가속에 맞춰져 있다면 이보다 훌륭한 대안은 존재하지 않는다. 이 패턴은 단순히 속도를 엄청나게 빨라지게 만들어 줄 뿐만 아니라, 10대의 서버 중 2~3대가 갑자기 화재로 타서 터지더라도 나머지 정상 서버 중 한 놈이 빛의 속도로 응답을 대신 메워주는 완벽한 ‘결함 허용 시스템’과 ‘고가용성 확장성’ 아키텍처를 별도의 자물쇠 감시 로직 없이 동시성 조립 라인 하나만으로 자연스럽게 동반 선사 해주기 때문이다.

[5-5] 처리량 제한(Rate Limiting)

대규모 분산 시스템 아키텍처 환경에서 시스템의 자원 고갈을 막고 안정성을 극한으로 끌어올리는 필수 방어 체계인 처리량 제한(속도 제한) 패턴에 대해 알아보겠다.

API 서비스를 개발하거나 연동해 본 경험이 있다면 특정 자원에 대한 접근 횟수를 단위 시간당 일정 수치로 제한하는 레이트 리미팅을 반드시 마주치게 된다.

이 제한의 대상이 되는 자원은 API 연결 수뿐만 아니라 디스크 읽기/쓰기, 네트워크 패킷, 심지어 특정 에러 발생 빈도 등 시스템의 모든 리소스가 될 수 있다.

(1) 시스템에 처리량 제한을 반드시 심어야 하는 이유

왜 모든 서비스는 무제한 접근을 허용하지 않고 굳이 처리량을 제한할까?

이유1: 보안 및 서비스 거부 공격(DDoS) 원천 차단

처리량 제한을 심는 순간 시스템을 노리는 수많은 공격 벡터가 단숨에 무력화된다.

만약 악의적인 유저가 자신의 컴퓨터 자원이 허용하는 최소 속도로 내 시스템에 요청을 폭격할 수 있다면, 순식간에 서버 디스크를 유령 로그나 가짜 요청으로 가득 채워 마비시킬 수 있다.

만약 로그 로테이션 설정이 미흡하다면, 공격자는 악성 행위를 저지른 뒤 엄청난 수의 무의미한 요청을 고속으로 발생시켜 자신들의 범죄 흔적이 담긴 로그 기록을 /dev/null 밖으로 밀어내 지워버릴 수도 있다. 무차별 대입(Brute-force) 공격이나 분산 서비스 거부(DDoS) 공격으로부터 시스템을 지키기 위한 최우선 방어벽이 바로 레이트 리미팅이다.

이유2: 다른 유저 보호 및 ‘죽음의 나선’ 예방

꼭 악의적인 공격이 아니더라도, 특정 적법한 유저가 비정상적으로 루프를 도는 버그 코드를 배포하거나 과도한 대량 연산을 수행하면 시스템 전체의 성능이 저하되어 같은 서버를 공유하는 다른 무고한 유저들이 피해를 보게 된다. 이 시스템 전체가 연쇄적으로 무너지는 ‘죽음의 나선’을 유발한다.

사용자는 기본적으로 “내가 시스템을 쓰는 공간은 안전하게 격리되어 있어서 다른 사람의 행동에 영향을 받지 않는다”는 심리적 모델을 가지고 서비스를 이용한다.

이 모델이 깨지면 유저들은 시스템 설계가 엉망이라고 느끼며 서비스를 이탈하게 된다.

이유3: 시스템 자가 공격(Self-DDoS) 방지

유저가 단 한 명뿐인 시스템일지라도 제한은 필요하다. 대다수의 시스템은 일반적인 예측 부하 범위 내에서만 정상 작동하도록 설계되어 있으므로, 한계를 벗어나는 극단적인 상황을 마주하면 완전히 다르게 오작동하기 시작한다.

예를 들어 갑작스러운 과부하로 인해 내부 네트워크 패킷이 누락되기 시작하면, 분산 데이터베이스의 정족수(Quorum)가 깨져 쓰기 연산이 전면 중단되고, 이로 인해 기존 요청들이 도미노처럼 실패하며, 실패한 요청들이 다시 재시도 폭탄을 생성하는 악순환이 발생한다. 시스템이 스스로를 DDoS 공격하는 좀비 상태가 되는 것이다.

(2) 토큰 버킷 알고리즘의 원리

레이트 리미팅을 구현할 때 전 세계적으로 가장 널리 쓰이는 표준 알고리즘은 토큰 버킷(Token Bucket) 알고리즘이다. 개념이 매우 직관적이고 코드로 구현하기도 쉽다.

원리는 다음과 같다.

- 시스템 자원을 1번 소모할 때마다 무조건 1개의 '액세스 토큰'이 필요하다고 가정한다. 가방에 토큰이 없으면 요청은 즉시 거절당하거나 대기열에 갇힌다.
- 이 토큰들을 담아두는 창고인 '버킷'이 존재한다. 이 버킷은 담을 수 있는 최대 토큰의 용량 한계인 깊이(d, Depth)를 가진다. 깊이가 5라면 토큰을 최대 5개까지만 보유할 수 있다.
- 버킷이 가득 찬 상태에서는 토큰 5개를 소모하는 5번의 연속적인 폭발적인 요청을 대기 시간 없이 즉시 처리할 수 있다. 이를 버스트(Burst, 순간 누적 허용량)라고 부른다.
- 토큰을 다 써버린 상태에서 6번째 요청이 들어오면 토큰이 채워질 때까지 요청은 차단된다.
- 이 버킷 창고에 토큰을 다시 채워 넣어주는 충전 속도를 충전율(r, Rate)이라고 부른다. 예를 들어 1초에 1개씩 충전되도록 설정할 수 있으며, 이 r값이 우리가 흔히 말하는 초당 처리량 제한 기준이 된다.

(3) Go 표준 라이브러리를 활용한 단일 레이트 리미터 구현

Go언어 진영에서는 준표준 라이브러리인 `golang.org/x/time/rate` 패키지를 통해 매우 정교한 토큰 버킷 리미터를 제공한다.

패키지의 핵심 함수인 `rate.NewLimiter(r, b)` 인터페이스를 활용하여 아무런 제한 장치가 없던 가상의 API 연결 구조체에 “초당 정확히 1건($r = 1$), 최대 버스트 용량 1칸($b = 1$)”의 정석적인 방어벽을 구축한 전체 코드를 실행해보겠다.

```
package main

import (
    "context" // 취소, 타임아웃 등을 전달하기 위한 Context
    "log"     // 로그 출력
    "os"      // Stdout 사용
    "sync"    // WaitGroup 사용
    "time"

    "golang.org/x/time/rate" // Rate Limiter
)

// API 서버 연결을 표현하는 구조체
type APIConnection struct {

    // 요청 속도를 제한하는 토큰 버킷 리미터
    //
    // *rate.Limiter
    // → 포인터 타입
    // → 여러 고루틴이 동일한 Limiter 공유
    rateLimiter *rate.Limiter
}
```

```

// APIConnection 생성 함수
func Open() *APIConnection {

    // &APIConnection
    // → 구조체 생성 후 주소 반환
    return &APIConnection{

        // rate.NewLimiter(r, b)
        //
        // r = 토큰 생성 속도
        // b = 버킷 최대 크기
        //
        // rate.Limit(1)
        // → 초당 토큰 1개 생성
        //
        // 1
        // → 버킷 최대 크기 1개
        //
        // 결과:
        // 초당 1번만 API 사용 가능
        rateLimiter: rate.NewLimiter(rate.Limit(1), 1),
    }
}

// 파일 읽기 API
func (a *APIConnection) ReadFile(ctx context.Context) error {

    // 토큰 획득 시도
    //
    // 토큰이 있으면:
    // 즉시 통과
    //
    // 토큰이 없으면:
    // 토큰 충전될 때까지 대기
    if err := a.rateLimiter.Wait(ctx); err != nil {

        // Context 취소
        // Context Timeout
        // 등의 상황
        return err
    }

    log.Printf("ReadFile API executed")

    return nil
}

// 주소 해석 API
func (a *APIConnection) ResolveAddress(ctx context.Context) error {

    // 위와 동일

```

```

    if err := a.rateLimiter.Wait(ctx); err != nil {
        return err
    }

    log.Printf("ResolveAddress API executed")

    return nil
}

func main() {

    // main 종료 직전에 실행
    defer log.Printf("Done.")

    // 로그를 콘솔에 출력
    log.SetOutput(os.Stdout)

    // 로그 출력 형식
    //
    // log.Ltime
    // → 시간 출력
    //
    // log.LUTC
    // → UTC 기준
    log.SetFlags(log.Ltime | log.LUTC)

    // APIConnection 생성
    apiConnection := Open()

    // WaitGroup 생성
    var wg sync.WaitGroup

    // 앞으로 20개의 작업을 기다리겠다고 선언
    //
    // 내부 카운터:
    // 0 → 20
    wg.Add(20)

    //-----
    // 파일 읽기 고루틴 10개 생성
    //-----

    for i := 0; i < 10; i++ {

        // 새로운 고루틴 생성
        go func() {

            // 고루틴 종료 시 카운터 감소
            defer wg.Done()

            // Context 생성
            //

```

```

        // 취소 없음
        // 타임아웃 없음
        ctx := context.Background()

        // API 호출
        //
        // 반환 에러는 무시
        _ = apiConnection.ReadFile(ctx)

        log.Printf("ReadFile finished")
    }()
}

//-----
// 주소 해석 고루틴 10개 생성
//-----

for i := 0; i < 10; i++ {

    go func() {

        defer wg.Done()

        ctx := context.Background()

        _ = apiConnection.ResolveAddress(ctx)

        log.Printf("ResolveAddress finished")
    }()
}

//-----
// 여기서 대기
//-----

// WaitGroup 카운터가
//
// 20
// ↓
// 19
// ↓
// ...
// ↓
// 0
//
// 이 될 때까지 block
wg.Wait()
}

```

참고: 위 코드를 로컬 환경에서 실행하려면 터미널에 `go get golang.org/x/time/rate` 명령어로 외부 패키지를 받아오거나, 아래 소스코드에 내장 뼈대를 부착하여 가동해야 한다. 여기서는 저장 가동시의 타임라인 타임스탬프 출력 결과를 완벽하게 명시하겠다.

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
22:08:30 ResolveAddress
22:08:31 ReadFile
22:08:32 ReadFile
22:08:33 ReadFile
22:08:34 ResolveAddress
22:08:35 ResolveAddress
22:08:36 ResolveAddress
22:08:37 ResolveAddress
22:08:38 ResolveAddress
22:08:39 ReadFile
22:08:40 ResolveAddress
22:08:41 ResolveAddress
22:08:42 ResolveAddress
22:08:43 ResolveAddress
22:08:44 ReadFile
22:08:45 ReadFile
22:08:46 ReadFile
22:08:47 ReadFile
22:08:48 ReadFile
22:08:49 ReadFile
22:08:49 Done.
```

제한 장치가 없을 때는 20개의 요청이 단 1나노초 만에 한꺼번에 쏟아져 들어와 하류 서버를 강타했었지만, 토큰 버킷 리미터를 장착한 뒤에는 출력 결과 타임스탬프를 보시다시피 매 초당 정확히 1초의 간격을 두고 한 건씩 차례대로 안전하게 처리되는 것을 볼 수 있다.

(4) 다중 계층 복합 처리량 제한 패턴

실무 프로덕션 환경에서는 방금 본 단일 리미터만으로는 세련된 비즈니스 요구사항을 충족하기 어렵다. 보통은 시스템 부하를 더 촘촘하게 제어하기 위해 “초당 최대 2번까지 순간 허용하되, 분당 총 누적 요청 수는 10번을 넘지 못하게 차단하라”와 같은 다중 계층 제한 아키텍처를 구사해야 한다.

이를 깔끔하게 구현하기 위해 개별 리미터들을 배열로 묶어 결합하고, 한 번 요청이 들어올 때마다 모든 리미터의 토큰을 연쇄적으로 차감 검사하는 복합형 MultiLimiter 패턴 전체 코드를 구현해 실행해보겠다.

```
package main

import (
    "context"
    "fmt"
    "log"
    "os"
    "sort"
    "sync"
    "time"
)

// [1] 토큰 버킷 구현
type Limit float64 // 초당 허용량을 표현하는 타입
```

```

type Limiter struct { // 토큰 버킷 상태 저장
    rate      float64 // 초당 생성되는 토큰 수
    burst     int // 버킷 최대 크기
    tokens     float64 // 현재 보유 중인 토큰 수
    lastRefreshed time.Time // 마지막으로 토큰을 충전한 시각
    mu        sync.Mutex // 여러 고루틴이 동시에 접근하는 것을 방지
}

// Limiter 생성
// r: 초당 생성 속도, b: 버킷 크기
func NewLimiter(r float64, b int) *Limiter {
    return &Limiter{
        rate: r,
        burst: b,

        // 시작 시 버킷 가득 채움
        tokens: float64(b),
        lastRefreshed: time.Now(),
    }
}

// 토큰 획득 함수
// 토큰이 있으면 즉시 통과
// 없으면 토큰이 생성될 때까지 대기
func (l *Limiter) Wait(ctx context.Context) error {
    // 토큰 버킷 상태 보호
    l.mu.Lock()
    defer l.mu.Unlock()

    // 현재 시각
    now := time.Now()

    // 마지막 갱신 이후 흐른 시간 계산
    elapsed := now.Sub(l.lastRefreshed).Seconds()
    l.lastRefreshed = now // 마지막 갱신 시각 업데이트

    // 흐른 시간만큼 토큰 충전
    // 예: rate = 2, elapsed=3초 => 6개 충전
    l.tokens += elapsed * l.rate

    // 버킷 크기 초과 방지
    if l.tokens > float64(l.burst) {
        l.tokens = float64(l.burst)
    }

    // 토큰이 1개 이상 있으면
    if l.tokens >= 1.0 {
        // 토큰 사용
        l.tokens -= 1.0
        // 즉시 통과
        return nil
    }
}

```

```

// 토큰이 부족하면 필요한 대기 시간을 계산해 강제 취침 블로킹
// 예: 현재 0.3개 => 0.7개 더 필요
neededTokens := 1.0 - l.tokens

// 필요한 토큰이 생성될 때까지 기다릴 시간 계산
sleepDuration := time.Duration(neededTokens / l.rate * float64(time.Second))
l.tokens = 0.0 // 토큰 전부 사용 처리

// 잠들기 전에 락 해제
l.mu.Unlock()
time.Sleep(sleepDuration)
l.mu.Lock()
// 깨어난 시점 기준으로 갱신
l.lastRefreshed = time.Now()
return nil
}
// 현재 제한 속도 반환
func (l *Limiter) Limit() Limit {
    return Limit(l.rate)
}

// 편의 함수
// 예: Per(2, time.Second) => 초당 2개
// 예: Per(10, time.Minute) => 분당 10개
func Per(eventCount int, duration time.Duration) Limit {
    return Limit(float64(eventCount) / duration.Seconds())
}

// =====
// RateLimiter 인터페이스
// Wait()
// Limit()
// 둘 다 구현하면 RateLimiter 취급
// =====
type RateLimiter interface {
    Wait(context.Context) error
    Limit() Limit
}

// 여러 Limiter를 합치는 구조체
type multiLimiter struct {
    limiters []RateLimiter
}

func MultiLimiter(limiters ...RateLimiter) *multiLimiter {
    // 최적화: 제한 수치가 가장 짝퍽한 리미터 순서대로 정렬하여 조기 필터링 효율 향상
    // 예: 분당 10개, 초당 2개 => 분당 10개가 먼저
    sort.Slice(limiters, func(i, j int) bool {
        return limiters[i].Limit() < limiters[j].Limit()
    })
    return &multiLimiter{limiters: limiters}
}

```



```

// 모든 Limiter를 통과해야 성공
func (l *multiLimiter) Wait(ctx context.Context) error {
    // 내가 가진 초단위, 분단위 모든 리미터 장벽의 Wait 통문을 순서대로 통과해야 최종 처리 승인함
    for _, limiter := range l.limiters {
        if err := limiter.Wait(ctx); err != nil {
            return err
        }
    }
    return nil
}

// 가장 엄격한 제한 반환
func (l *multiLimiter) Limit() Limit {
    return l.limiters[0].Limit()
}

// =====
// API 연결 객체
// =====
type APIConnection struct {
    // 단일 Limiter 또는 MultiLimiter 가능
    rateLimiter RateLimiter
}

// =====
// APIConnection 생성
// =====
func Open() *APIConnection {
    // 다중 레이어 주입: 초당 2개 제한 버킷(용량 1) + 분당 10개 제한 버킷(용량 10) 결합
    secondLimit := NewLimiter(Per(2, time.Second), 1)
    minuteLimit := NewLimiter(Per(10, time.Minute), 10)

    return &APIConnection{
        // 두 제한 결합
        rateLimiter: MultiLimiter(secondLimit, minuteLimit),
    }
}

// 파일 읽기 API
func (a *APIConnection) ReadFile(ctx context.Context) error {
    // 제한 통과 대기
    if err := a.rateLimiter.Wait(ctx); err != nil {
        return err
    }
    return nil
}

// 주소 해석 API
func (a *APIConnection) ResolveAddress(ctx context.Context) error {
    if err := a.rateLimiter.Wait(ctx); err != nil {

```

```

        return err
    }
    return nil
}

func main() {
    defer log.Printf("Done.")
    log.SetOutput(os.Stdout)
    log.SetFlags(log.Ltime | log.LUTC)

    apiConnection := Open()
    var wg sync.WaitGroup
    wg.Add(20)

    for i := 0; i < 10; i++ {
        go func() {
            defer wg.Done()
            _ = apiConnection.ReadFile(context.Background())
            log.Printf("ReadFile")
        }()
    }
    for i := 0; i < 10; i++ {
        go func() {
            defer wg.Done()
            _ = apiConnection.ResolveAddress(context.Background())
            log.Printf("ResolveAddress")
        }()
    }

    wg.Wait()
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

22:46:10 ResolveAddress
22:46:10 ReadFile
22:46:11 ReadFile
22:46:11 ReadFile
22:46:12 ReadFile
22:46:12 ReadFile
22:46:13 ReadFile
22:46:13 ReadFile
22:46:14 ReadFile
22:46:14 ReadFile
22:46:16 ResolveAddress
22:46:22 ResolveAddress
22:46:28 ReadFile
22:46:34 ResolveAddress
22:46:40 ResolveAddress
22:46:46 ResolveAddress
22:46:52 ResolveAddress
22:46:58 ResolveAddress

```

```
22:47:04 ResolveAddress
22:47:10 ResolveAddress
22:47:10 Done.
```

출력 타임라인 결과를 정밀 분석해보자.

- 초반 스퍼트 구간(10번째 요청까지): 22:46:10초부터 22:46:14초까지 정확하게 초당 2건씩의 속도로 쾌적하고 시원하게 응답이 튀어나온다. 초당 리미터 제한 장벽 내에서 안정적으로 최고 속도를 낸다.

- 지연 정제 구간(11번째 요청부터): 11번째 요청이 실행되는 22:46:16초 지점부터 갑자기 다음 요청들이 터지는 간격이 정확히 '6초' 간격으로 둔탁하게 툭 꺾이며 정제되는 것을 볼 수 있다. 분당 허용 총량인 10개의 버킷 토큰을 초반에 순식간에 다 소모해 버렸기 때문에, 이제 분당 리미터가 흘러보내 주는 슬라이딩 윈도우 토큰 충전 속도(60초/10개 = 6초에 1개)에 묶여 시스템 제어 모드가 강제 전환된 것이다.

이처럼 멀티 리미터 패턴을 활용하면 거친 부하 폭격 속에서도 하위 인프라 시스템의 심장 박동을 완벽하게 요리하고 통제할 수 있다.

(5) 시간, 디스크, 네트워크를 넘나드는 다차원 자원 제한 설계

한 단계 더 나아가 실무에서는 단순히 API 호출 수(시간 차원)뿐만 아니라 디스크 드라이브 읽기 속도, 네트워크 대역폭 패킷 전송량 등 서로 다른 물리 차원의 자원들을 교차 결합하여 통제해야 한다. 파일 읽기 연산은 'API 제한 + 디스크 제한'을 통과하게 만들고, 네트워크 주소 변환 연산은 'API 제한 + 네트워크 제한'의 장벽을 결합 통과하게 설계하는 다차원 입체 아키텍처 전체 코드를 확인해보자.

```
package main

import (
    "context"
    "fmt"
    "log"
    "os"
    "sort"
    "sync"
    "time"
)

// =====
// 초당 허용량을 나타내는 타입
//
// 예)
// Per(2, time.Second)
// => Limit(2.0)
//
// 의미:
// 초당 토큰 2개 생성
// =====
type Limit float64

// =====
```

```

// 토큰 버킷 리미터
// =====
type Limiter struct {

    // 초당 생성되는 토큰 수
    rate float64

    // 버킷 최대 크기
    burst int

    // 현재 보유 토큰 수
    tokens float64

    // 마지막 충전 시각
    lastRefreshed time.Time

    // 동시 접근 보호용 락
    mu sync.Mutex
}

// =====
// 리미터 생성
// =====
func NewLimiter(r Limit, b int) *Limiter {
    return &Limiter{

        // 초당 생성량
        rate: float64(r),

        // 최대 저장량
        burst: b,

        // 시작 시 버킷 가득 채움
        tokens: float64(b),

        // 현재 시각 기록
        lastRefreshed: time.Now(),
    }
}

// =====
// 토큰 획득
//
// 토큰이 있으면 즉시 통과
// 없으면 생성될 때까지 대기
// =====
func (l *Limiter) Wait(ctx context.Context) error {

    // 토큰 상태 보호
    l.mu.Lock()

    // 함수 종료 시 락 해제

```

```

defer l.mu.Unlock()

// 현재 시각
now := time.Now()

// 마지막 충전 이후 경과 시간
elapsed := now.Sub(l.lastRefreshed).Seconds()

// 마지막 충전 시각 갱신
l.lastRefreshed = now

// 경과 시간만큼 토큰 충전
l.tokens += elapsed * l.rate

// 버킷 초과 금지
if l.tokens > float64(l.burst) {
    l.tokens = float64(l.burst)
}

// 토큰이 있으면
if l.tokens >= 1.0 {

    // 1개 사용
    l.tokens -= 1.0

    // 통과
    return nil
}

// 부족한 토큰 계산
//
// 예)
// 0.3개 있으면
// 0.7개 더 필요
neededTokens := 1.0 - l.tokens

// 필요한 토큰 생성 시간 계산
sleepDuration :=
    time.Duration(
        neededTokens /
        l.rate *
        float64(time.Second),
    )

// 토큰 사용 처리
l.tokens = 0.0

// 잠자는 동안 락 점유 방지
l.mu.Unlock()

// 토큰 생성될 때까지 대기
time.Sleep(sleepDuration)

```

```

        // 다시 락 획득
        l.mu.Lock()

        // 마지막 충전 시각 갱신
        l.lastRefreshed = time.Now()

        return nil
    }

    // =====
    // 현재 제한 속도 반환
    // =====
    func (l *Limiter) Limit() Limit {
        return Limit(l.rate)
    }

    // =====
    // 편의 함수
    //
    // Per(2,time.Second)
    // => 초당 2개
    //
    // Per(10,time.Minute)
    // => 분당 10개
    // =====
    func Per(eventCount int, duration time.Duration) Limit {

        return Limit(

            // 이벤트 수
            float64(eventCount) /

            // 초 단위 시간
            duration.Seconds(),

        )
    }

    // =====
    // 리미터 인터페이스
    //
    // Wait()
    // Limit()
    //
    // 둘 다 구현하면 RateLimiter 취급
    // =====
    type RateLimiter interface {

        // 토큰 획득
        Wait(context.Context) error

        // 제한량 반환

```

```

    Limit() Limit
}

// =====
// 여러 리미터를 합치는 구조체
// =====
type multiLimiter struct {

    // 리미터 목록
    limiters []RateLimiter
}

// =====
// 여러 리미터 결합
// =====
func MultiLimiter(limiters ...RateLimiter) *multiLimiter {

    // 가장 빠른 리미터부터 정렬
    sort.Slice(limiters, func(i, j int) bool {

        return limiters[i].Limit() <
            limiters[j].Limit()

    })

    return &multiLimiter{
        limiters: limiters,
    }
}

// =====
// 모든 리미터 통과해야 성공
// =====
func (l *multiLimiter) Wait(ctx context.Context) error {

    for _, limiter := range l.limiters {

        // 하나라도 실패하면 종료
        if err := limiter.Wait(ctx); err != nil {
            return err
        }

    }

    return nil
}

// =====
// 가장 엄격한 제한 반환
// =====
func (l *multiLimiter) Limit() Limit {

    return l.limiters[0].Limit()
}

```

```

// =====
// API 연결 객체
// =====
type APIConnection struct {

    // API 전체 제한
    apiLimit RateLimiter

    // 디스크 제한
    diskLimit RateLimiter

    // 네트워크 제한
    networkLimit RateLimiter
}

// =====
// APIConnection 생성
// =====
func Open() *APIConnection {

    return &APIConnection{

        // -----
        // API 전체 제한
        //
        // 초당 2개
        // 분당 10개
        // -----
        apiLimit: MultiLimiter(
            NewLimiter(
                Per(2, time.Second),
                2,
            ),
            NewLimiter(
                Per(10, time.Minute),
                10,
            ),
        ),

        // -----
        // 디스크 제한
        //
        // 초당 1개
        // -----
        diskLimit: MultiLimiter(
            NewLimiter(
                Limit(1),
                1,
            ),
        ),
    }
}

```



```

// -----
// 네트워크 제한
//
// 초당 3개
// -----
networkLimit: MultiLimiter(
    NewLimiter(
        Per(3, time.Second),
        3,
    ),
),
}
}

// =====
// 파일 읽기
//
// API 제한 +
// 디스크 제한
//
// 둘 다 통과해야 함
// =====
func (a *APIConnection) ReadFile(
    ctx context.Context,
) error {

    err :=
        MultiLimiter(
            a.apiLimit,
            a.diskLimit,
        ).Wait(ctx)

    if err != nil {
        return err
    }

    return nil
}

// =====
// 주소 조회
//
// API 제한 +
// 네트워크 제한
//
// 둘 다 통과해야 함
// =====
func (a *APIConnection) ResolveAddress(
    ctx context.Context,
) error {

    err :=

```

```

        MultiLimiter(
            a.apiLimit,
            a.networkLimit,
        ).Wait(ctx)

    if err != nil {
        return err
    }

    return nil
}

func main() {

    // 프로그램 종료 시 출력
    defer log.Printf("Done.")

    log.SetOutput(os.Stdout)

    log.SetFlags(
        log.Ltime |
        log.LUTC,
    )

    // API 객체 생성
    apiConnection := Open()

    // 고루틴 종료 대기
    var wg sync.WaitGroup

    // 총 20개 작업 예정
    wg.Add(20)

    // =====
    // ReadFile 10개 실행
    // =====
    for i := 0; i < 10; i++ {

        go func() {

            defer wg.Done()

            // 제한 검사
            _ = apiConnection.ReadFile(
                context.Background(),
            )

            log.Printf("ReadFile")

        }()
    }

    // =====

```

```

// ResolveAddress 10개 실행
// =====
for i := 0; i < 10; i++ {

    go func() {

        defer wg.Done()

        // 제한 검사
        _ = apiConnection.ResolveAddress(
            context.Background(),
        )

        log.Printf("ResolveAddress")

    }()

}

// 모든 고루틴 종료 대기
wg.Wait()
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

01:40:15 ResolveAddress
01:40:15 ReadFile
01:40:16 ReadFile
01:40:17 ResolveAddress
01:40:17 ResolveAddress
01:40:17 ReadFile
01:40:18 ResolveAddress
01:40:18 ResolveAddress
01:40:19 ResolveAddress
01:40:19 ResolveAddress
01:40:21 ResolveAddress
01:40:27 ResolveAddress
01:40:33 ResolveAddress
01:40:39 ReadFile
01:40:45 ReadFile
01:40:51 ReadFile
01:40:57 ReadFile
01:41:03 ReadFile
01:41:09 ReadFile
01:41:15 ReadFile
01:41:15 Done.

```

(6) 결론

복합 다차원 처리량 제한 지표 결과를 확인해보자. 각 고루틴 일꾼들이 수십 마리씩 비동기식으로 동시에 도망치며 날뛰는 와중에도, 내가 정의해둔 물리 자원의 고유 한계치(디스크 초당 1회, 네트워크 초당 3회, 전체 API 상한선)의 교차점 수학 법칙을 정확히 계산해 내어 자원 파괴 없이 물줄기가 정돈되어 배출되는 것을 볼 수 있다.

처리량 제한은 시스템 인프라 구축의 사치품이 아니라, 예상치 못한 트래픽 폭풍과 버그성 무한 재시도 지옥 속에서 서버를 생존시키는 가장 원초적이고 강력한 면역 체계이다. 시스템 아키텍처를 설계할 때 진입 장벽 단계부터 이 토큰 버킷 기반의 레이어 리미팅 방어선들을 촘촘하게 구축하여, 구 어떤 부하 환경에서도 무너지지 않는 예측 가능하고 단단한 고성능 동시성 시스템을 완성해보자.

[5-6] 비정상 고루틴 자가 치유(Healing Unhealthy Goroutines)

Go언어의 고성능 대규모 동시성 아키텍처 환경에서 시스템의 연속성을 보장하는 최첨단 자가 치유 기술인 비정상 고루틴 자가 치유 패턴에 대해 알아보겠다.

데몬(Daemons) 프로그램처럼 24시간 내내 죽지 않고 작동해야 하는 장기 실행 프로세스 시스템에서는 백그라운드에서 상시 대기하며 일하는 고루틴 일꾼들을 대거 띄워두는 것이 일반적이다. 이 일꾼 고루틴들은 대개 평소에는 블로킹(대기) 상태로 얌전히 서 있다가, 데이터가 들어오는 순간 깨어나 연산을 수행하고 결과를 다음 단계로 토스한다.

문제는 이 고루틴들이 외부 웹 서비스 API 호출이나 파일 시스템 감시처럼 “우리가 완벽하게 제어할 수 없다는 불안정한 외부 자원”에 무겁게 의존하고 있을 때 발생한다. 외부 네트워크 지연이나 시스템 예외 버그로 인해 고루틴이 스스로 회복 불가능한 먹통 상태에 갇히는 대참사가 실무에서는 빈번하게 터진다.

이때 4장에서 배운 ‘관심사의 분리’ 철학을 대입해 보면, “열심히 비즈니스 연산을 수행하는 일꾼 고루틴에게 자기가 고장났을 때 스스로를 수리하고 부활시키는 복잡한 무덤 연산의 책임까지 지우는 것은 잘못된 설계”라는 정답에 도달한다.

시스템의 영구 생존을 위해서는 백그라운드 일꾼 고루틴들의 컨디션을 삼엄하게 감시하다가, 맥박이 멈추는 즉시 강제로 심장 충격을 가해 재시작(Restart)시켜 주는 별도의 자가 치유 오케스트레이션 엔진을 구축해야 한다.

이 사상은 함수형 언어의 대명사인 Erlang의 슈퍼바이저 트리(Supervision Tree) 자가 치유 메커니즘과 완벽하게 일맥상통하는 기술이다.

(1) 관리자와 피관리자: 스튜어드와 워드 아키텍처

고루틴의 건강 상태를 진단하고 수리하기 위해, 앞서 배운 하트비트 맥박 신호를 관문 도구로 활용한다.

동시성 아키텍처에서는 이 자가 치유 시스템의 주역을 다음과 같이 명명한다.

- 스튜어드(Steward, 관리자 고루틴)

피관리자 고루틴의 생존 하트비트를 실시간으로 모니터링하며, 타임아웃 기한 내에 맥박이 뛰지 않으면 해당 고루틴을 강제로 사살하고 부활시키는 책임을 진다.

- 워드(Ward, 피관리자 고루틴)

스튜어드의 감시 하에 들어가 묵묵히 비즈니스 연산을 수행하되, 자기가 살아있음을 증명하는 하트비트 맥박을 주기적으로 뱉어내야 하는 임무를 띤다.

스튜어드가 고장난 워드를 언제든 새로 생성할 수 있도록, 워드를 시동하는 함수 포인터(`startGoroutineFn`)의 주소 참조 열쇠를 스튜어드에게 주입해야 한다. 정석적인 스튜어드 감시 오케스트레이터의 엔진 소스코드를 생략 없이 구현한다.

```

package main

import (
    "context"
    "log"
    "os"
    "sync"
    "time"
)

// 헬퍼 함수: 두 개의 done 취소 채널 중 하나라도 먼저 닫히면 중단 신호를 통과시키는 or 채널
// 패턴
func or(channels ...<-chan interface{}) <-chan interface{} {
    switch len(channels) {
    case 0:
        return nil
    case 1:
        return channels[0]
    }
    orDone := make(chan interface{})
    go func() {
        defer close(orDone)
        switch len(channels) {
        case 2:
            select {
            case <-channels[0]:
            case <-channels[1]:
            }
        default:
            select {
            case <-channels[0]:
            case <-channels[1]:
            case <-channels[2]:
            case <-or(append(channels[3:], orDone)...):
            }
        }
    }()
    return orDone
}

// 규칙 1: 감시 대상이 될 피관리자(Ward) 고루틴 함수가 반드시 준수해야 하는 표준 인터페이스
// 서명 정의
type startGoroutineFn func(
    done <-chan interface{},
    pulseInterval time.Duration,
) (heartbeat <-chan interface{})

// 규칙 2: 고장 난 워드를 소생시킬 권한을 가진 스튜어드(Steward) 팩토리 오케스트레이터 구현
newSteward := func(
    timeout time.Duration,
    // 감시할 워드(worker)

```

```

startGoroutine startGoroutineFn,
) startGoroutineFn {
    // 재미있게도 스튜어드 자신 또한 똑같은 서명의 startGoroutineFn을 반환하므로,
    // 스튜어드 위에 더 거대한 스튜어드를 얹어 다중 계층 관리자 트리(Supervision Tree)를
    // 조립할 수 있습니다.
    return func(
        done <-chan interface{},
        pulseInterval time.Duration,
    ) <-chan interface{} {
        heartbeat := make(chan interface{})

        go func() {
            defer close(heartbeat)
            // 현재 Ward 종료용 채널
            var wardDone chan interface{}
            // 현재 Ward의 heartbeat 채널
            var wardHeartbeat <-chan interface{}

            // 워드 고루틴을 깨끗하게 시동하는 내부 격리 클로저 정의
            startWard := func() {
                wardDone = make(chan interface{})
                // 부모 스튜어트가 멈추거나, 혹은 관리자가 자식을 저격 취소하
                // 고 싶을 때
                // 양쪽 신호 모두에 연동되도록 or 채널 방어벽으로 취소 통로를
                // 묶어 주입합니다.
                wardHeartbeat = startGoroutine(or(wardDone, done), timeout/
2)

            }

            // 첫 번째 워드 일꾼 시동!
            startWard()
            pulse := time.Tick(pulseInterval)

            for {
                // 매 루프마다 워드의 맥박을 기다려줄 단축 시한부 타이머 가동
                timeoutSignal := time.After(timeout)

                for {
                    select {
                        case <-pulse: // 스튜어드 자신도 살아있음을 상류 감시
                            // 수집 성공: 고장 나지 않고 자식 워드의 하트비
                            // 내부 대기 Select문을 즉시 깨부수고 바깥쪽 감
                            // 시 루프로 넘어가 타이머 시계를 초기화합니다.
                            continue topMonitorLoop
                        case <-timeoutSignal:
                    }
                }
            }
        }()
    }
}

```

탭에 보고하기 위해 맥박 송출
 트 맥박이 정상 인입되면

```

// 비상 상황 발동: 약속된 타임아웃 기한 동안 자
식의 맥박이 단 한 번도 뛰지 않았다면
restartng")

워드 자식에게 사살(취소) 신호 발송
close(wardDone) // 1. 고장 나서 좀비가 된 기존
고루틴을 백그라운드에 즉시 재부팅 소생!
startWard() // 2. 새로운 깨끗한 복사본 워드

continue topMonitorLoop
case <-done:
    return
}
}
topMonitorLoop: // 레이블을 활용해 이중 루프 스케줄링 흐름 제어
}
}0
return heartbeat
}
}

```

(2) 무책임한 일꾼 고루틴 저격 테스트

이제 우리가 만든 스튜어드 관리자 엔진의 성능을 검증해보자.

백그라운드에서 시동되자마자 만천하에 “ward: Hello, I’m irresponsible!”이라는 거대한 무책임 로그만 딱 한 줄 찍은 뒤, 생존 하트비트 맥박은 단 한 번도 전송하지 않은 채 공짜 밥만 먹으며 뺏어버리는 악성 좀비 워드 고루틴을 매칭해 가동해보겠다. 생략 없는 전체 제어 코드를 실행해 확인해보자.

```

package main

import (
    "log"
    "os"
    "time"
)

// 앞서 구현한 or 채널 및 newSteward 엔진 로직이 상단에 내장되어 작동함 예시

func main() {
    log.SetOutput(os.Stdout)
    log.SetFlags(log.Ltime | log.LUTC)

    // 1. 하트비트를 전혀 안 보내고 멍하니 락만 걸려 서 있는 최악의 일꾼 고루틴(Ward) 정
    의
    doWork := func(done <-chan interface{}, _ time.Duration) <-chan interface{} {
        log.Println("ward: Hello, I'm irresponsible!")
        go func() {

```



```

        <-done // 오직 메인 흐름이 취소 채널을 완전히 닫아줄 때까지 평생 대기
        상태로 누워있음
        log.Println("ward: I am halting.")
    }()
    return nil // 하트비트 채널 자체를 아예 누락해서 상류에 안 돌려줌
}

// 2. 무책임한 doWork 고루틴을 4초짜리 정밀 타임아웃 감시 필터망을 가진 스튜어드
팩토리로 포장 결합
doWorkWithSteward := newSteward(4*time.Second, doWork)
done := make(chan interface{})

// 3. 9초 뒤 전체 데몬 메인 시스템이 안전하게 종료되도록 타이머 예약 세팅
time.AfterFunc(9*time.Second, func() {
    log.Println("main: halting steward and ward.")
    close(done)
})

// 4. 스튜어드 엔진 가동 및 생존 맥박 순회 루프 실행
for range doWorkWithSteward(done, 4*time.Second) {
}
log.Println("Done")
}

```

위 자가 치유 시뮬레이션 시스템 코드를 실행하면 다음과 같은 정교한 타임라인 로그 결과가 출력된다.

```

18:28:07 ward: Hello, I'm irresponsible!
18:28:11 steward: ward unhealthy; restarting
18:28:11 ward: Hello, I'm irresponsible!
18:28:11 ward: I am halting.
18:28:15 steward: ward unhealthy; restarting
18:28:15 ward: Hello, I'm irresponsible!
18:28:15 ward: I am halting.
18:28:16 main: halting steward and ward.
18:28:16 ward: I am halting.
18:28:16 Done

```

타임라인 처리 지표를 가만히 뜯어보자. 아키텍처가 소름 돋을 정도로 완벽하고 맑게 동작하고 있다.

- 18:28:07초: 무책임한 첫 번째 자식 워드가 기세 좋게 태어났다. 하지만 심장이 뛰지 않는다.
- 18:28:11초(정확히 4초 뒤): 스튜어드 감시 타워가 자식의 심정지 상태를 즉시 간파하고 “steward: ward unhealthy; restarting” 경보 총성을 울렸다. 곧바로 1번 자식에게 사살 신호를 보낸 구조체 메모리 롤백 탈출(ward: I am halting.)을 유도함과 동시에, 완전히 새로운 2번 복사본 자식 워드를 그 즉시 제자리에 수평 부활시켜 연산 스트림의 연속성을 확보했다.
- 18:28:15초(정확히 4초 뒤): 새로 태어난 2번 자식 역시 정신을 못차리고 맥박을 안보내자. 스튜어드는 한 치의 오차도 없이 4초의 유예 기간이 만료되는 순간 즉시 2번자식을 사살하고 3번 복사본 자식을 새로 창조해냈다.

24시간 가동되는 엔터프라이즈 서버 환경에서 특정 고루틴이 자원 락에 걸려 바보가 되더라도, 외부 인프라 관리자의 수동 개입 없이 시스템 스스로가 밀리초 단위로 자식을 청소하고 소생시키는 자가 치유의 기적이 완성된 것이다.

(3) 클로저와 채널 브리징을 통한 실무형 복잡 데이터 스트림 결합 기술

방금 보신 기본형 워드 모델은 매개변수도 없고 반환 데이터도 없는 너무 장난감 같은 구조였다. 실무에서는 “고루틴 일꾼이 계속 죽고 새로 부활하며 채널 주소가 수시로 바뀌는 혼란스러운 인프라 대격변 상황 속에서도, 최종 하류의 소비자는 아무런 끔김 없이 맑고 순수한 단일 데이터 물줄기를 공급받아야 하는” 정교한 비즈니스 스펙을 구현해야 한다.

이 문제를 풀기 위해 앞서 배운 ‘클로저(Closure)의 자원 가두기 기술’과 여러 개의 채널 수송관을 기하학적으로 일렬로 이어 붙이는 ‘bridge-채널 패턴’을 결합 아키텍처로 총동원해야 한다.

정수 배열 슬라이스를 받아 유통하다가, 예러 신호의 마이너스 음수(-1)를 만나는 순간 시스템 컨디션 난조로 판정하고 스스로 죽어버리는 실무형 워드와 스튜어드 결합 전체 코드를 가동해보겠다.

```
package main

import (
    "fmt"
    "log"
    "os"
    "sort"
    "sync"
    "time"
)

// [앞서 마스터한 bridge 채널 및 or 채널 뼈대 로직 장착 레이어]
func bridge(done <-chan interface{}, chanStream <-chan (<-chan interface{})) <-chan
interface{} {
    valStream := make(chan interface{})
    go func() {
        defer close(valStream)
        for {
            var stream <-chan interface{}
            select {
            case maybeStream, ok := <-chanStream:
                if ok == false { return }
                stream = maybeStream
            case <-done: return
            }
            for val := range stream {
                select {
                case valStream <- val:
                case <-done:
                }
            }
        }
    }()
    return valStream
}
```

```

func or(channels ...<-chan interface{}) <-chan interface{} {
    if len(channels) == 0 { return nil }
    if len(channels) == 1 { return channels[0] }
    orDone := make(chan interface{})
    go func() {
        defer close(orDone)
        if len(channels) == 2 {
            select {
            case <-channels[0]:
            case <-channels[1]:
            }
            return
        }
        select {
        case <-channels[0]:
        case <-channels[1]:
        case <-channels[2]:
        case <-or(append(channels[3:], orDone)...):
        }
    }()
    return orDone
}

```

```

type startGoroutineFn func(<-chan interface{}, time.Duration) <-chan interface{}

```

```

func newSteward(timeout time.Duration, startGoroutine startGoroutineFn) startGoroutineFn
{
    return func(done <-chan interface{}, pulseInterval time.Duration) <-chan interface{} {
        heartbeat := make(chan interface{})
        go func() {
            defer close(heartbeat)
            var wardDone chan interface{}
            var wardHeartbeat <-chan interface{}
            startWard := func() {
                wardDone = make(chan interface{})
                wardHeartbeat = startGoroutine(or(wardDone, done), timeout/
2)
            }
            startWard()
            pulse := time.Tick(pulseInterval)
            for {
                timeoutSignal := time.After(timeout)
                for {
                    select {
                    case <-pulse:
                        select {
                        case heartbeat <- struct{}{}:
                        default:
                        }
                    case <-wardHeartbeat:
                        goto topLoop
                    }
                }
            }
        }()
    }
}

```

```

                                case <-timeoutSignal:
                                    log.Println("steward: ward unhealthy;
restarting")
                                    close(wardDone)
                                    startWard()
                                    goto topLoop
                                case <-done:
                                    return
                                }
                            }
                        topLoop:
                    }
                }()
            return heartbeat
        }
    }

func take(done <-chan interface{}, valueStream <-chan interface{}, num int) <-chan
interface{} {
    takeStream := make(chan interface{})
    go func() {
        defer close(takeStream)
        for i := 0; i < num; i++ {
            select {
            case <-done: return
            case takeStream <- <-valueStream:
            }
        }
    }()
    return takeStream
}

// =====
// 고급 동적 상태 결합 실무형 워드 생성기 함수
// =====
func main() {
    log.SetFlags(log.Ltime | log.LUTC)
    log.SetOutput(os.Stdout)

    doWorkFn := func(
        done <-chan interface{},
        intList ...int,
    ) (startGoroutineFn, <-chan interface{}) {
        // 아키텍처 핵심 1: 부활할 때마다 채널 주소가 파편화되는 문제를
        // 하나의 심리스한 물줄기로 묶어줄 대형 수송관 채널의 채널(chanStream)과 브
리징 구축
        intChanStream := make(chan (<-chan interface{}))
        intStream := bridge(done, intChanStream)

        // 스튜어트가 모니터링하며 언제나 무한 재시도 호출할 진짜 워드 껍데기 함수
정의
        doWork := func(

```

```

done <-chan interface{},
pulseInterval time.Duration,
) <-chan interface{} {
// 매번 부활할 때마다 새롭게 독립 개설되는 이번 고루틴 생명주기 전용
로컬 데이터 채널들
localIntStream := make(chan interface{})
heartbeat := make(chan interface{})

go func() {
defer close(localIntStream)

// 아키텍처 핵심 2: 새로 개설된 내 전용 로컬 채널 수송관 통로
주소 자체를
// 상류 브리징 총괄 본부인 intChanStream 속으로 쏘아 올려 대
기열에 등록시킵니다.
select {
case intChanStream <- localIntStream:
case <-done:
return
}

pulse := time.Tick(pulseInterval)
for {
// 주입된 비즈니스 데이터 슬라이스 배열을 정직하게 순
회 연산 처리
valueLoop:
for _, intVal := range intList {
// 인위적 버그 감지 지점: 음수 데이터를 만나면
시스템 컨디션 난조로 판정
if intVal < 0 {
log.Printf("negative value: %v\n",
intVal)
return // 하트비트를 끊고 고루틴 함수를
자폭 종료(Return)시켜 관리자를 호출합니다.
}

for {
select {
case <-pulse:
select {
case heartbeat <- struct{}{}:
default:
}
case localIntStream <- intVal: // 하류 브
리징 파이프라인으로 자원 무사 전송 성공 시
로 패스 이동
continue valueLoop // 다음 숫자

case <-done:
return
}
}
}
}

```

```

        }
    }()

    return heartbeat
}

return doWork, intStream
}

done := make(chan interface{})
defer close(done)

// 의도적으로 중간에 에러 폭탄인 마이너스 일(-1) 정수형 데이터를 슬라이스 중간에 매
립 주입
doWork, intStream := doWorkFn(done, 1, 2, -1, 3, 4, 5)

// 워드가 터지자마자 번개처럼 감지하도록 1밀리초짜리 극단적인 정밀 초단축 감시망 장
착
doWorkWithSteward := newSteward(1*time.Millisecond, doWork)
doWorkWithSteward(done, 1*time.Hour)

// 최종 하루 소비자는 상류에서 고루틴이 백 번 죽고 부활하든 상관없이,
// 오직 투명하게 정제된 intStream 단 하나만을 바라보며 최종 6개의 연산 결과를 성공
정수합니다.
for intVal := range take(done, intStream, 6) {
    fmt.Printf("Received: %v\n", intVal)
}
}

```

위 코드를 실행하면 다음과 같은 최종 결과가 출력된다.

```

Received: 1
23:25:33 negative value: -1
Received: 2
23:25:33 steward: ward unhealthy; restarting
Received: 1
23:25:33 negative value: -1
Received: 2
23:25:33 steward: ward unhealthy; restarting
Received: 1
23:25:33 negative value: -1
Received: 2

```

(4) 자가 치유 아키텍처 가동 시 주의해야 할 상태 보존 법칙

지표 출력 결과를 마주해보겠다. 최종 소비자 레벨인 메인 함수에서는 단 한 번의 끊임이나 런타임 캐스팅 패닉 없이 Received: 1, Received: 2라는 데이터 값들이 아주 맑고 안정적으로 연속 징수되는 기적을 보여주고 있다. 중간중간 섞여 나오는 로그를 보면 워드가 음수 폭탄을 밟고 장렬히 자폭할 때마다 스텔워드 소생 엔진이 빛의 속도로 일꾼을 무한 재부팅해 주며 시스템을 살려내고 있다.

하지만 여기서 한 가지 명심해야 할 중요한 시스템 아키텍처 부작용 지표가 보인다. 최종 유저가 받아본 숫자를 가만히 보니 1, 2, 1, 2, 1, 2 만 무한 반복해서 받아보고 있고, 정작 음수 뒤편에 숨겨져 있던 소중한 비즈니스 자원 데이터인 3, 4, 5 숫자는 영원히 구경조차 못하고 정체되어 있다.

그 이유는 현재 우리의 피관리자 워드가 부활할 때마다 아무런 과거 기억이 없는 순수한 제로 상태(Scratch)에서 맨 처음부터 배열 순회를 다시 시작하는 ‘무상태(Stateless)’ 구조로 설계되어 있기 때문이다.

내 시스템이 중복 데이터 인입에 무척 극도로 예민한 금융성 비즈니스 환경이라면 이런 무상태 자가 치유는 데이터 오염을 유발하므로 큰 독이 될 수 있다.

이 문제를 해결하기 위해서는 워드가 재시작되더라도 자기가 관거에 어디까지 일했는지 그 책임을 보존해 주는 상태 보존형 워드 아키텍처로 슬라이스 포인터를 보수해 주어야 한다.

// 기존의 무상태 정적 순회 방식 (맨 처음 인덱스부터 무조건 다시 읽음):

```
valueLoop:
for _, intVal := range intList {
    // ...
}
```

// 최적화 완료된 상태 보존형 동적 파괴 방식 (읽은 족족 원본 배열 슬라이스 주머니를 꺼아냄):

```
valueLoop:
for {
    if len(intList) == 0 { return }
    intVal := intList[0] // 언제나 맨 앞대자리 값만 읽음
    intList = intList[1:] // 읽자마자 클로저가 쥐고 있는 원본 배열 스펙 자체를 칼로 잘라 한 칸씩
    전진시킵니다!
    // ... 이 구조를 도입하면 부활하더라도 자기가 멈췄던 그 다음 지점부터 깔끔하게 연산을 이어
    나갑니다.
}
```

물론 이 상태 보존형 구조를 도입하더라도, 만약 에러의 원인이 되는 악성 원인 데이터 객체(-1) 자체가 배열 속에 지워지지 않고 영원히 박혀 있다면, 부활하자마자 또 음수를 밟고 터지고 부활하자마자 또 터지는 무한 패닉 지옥에 갇히게 되므로, 실무에서는 스텔워드 관리자 엔진 내부에 “연속 재시작 실패 횟수가 통산 5회가 넘어가면, 시스템 전체의 마비를 막기 위해 소생술을 과감히 포기하고 상류에 심각한 시스템 장애 리포트를 던진 뒤 스텔워드 자신도 동반 안전 종료하라”와 같은 강력한 서킷 브레이커 차단기 규칙을 임베디드하여 결합 설계하는 것이 정석이다.

(5) 결론

비정상 고루틴 자가 치유 패턴은 Go언어가 자랑하는 저수준 동시성 도구(채널, 고루틴, 선택트)들의 기하학적 규칙들을 한데 모아 대규모 클라우드 아키텍처의 영구 생존권을 확보하는 최고 등급의 아키텍처 마스터피스 기술이다.

내가 만든 시스템이 사람이 보지 않는 킥킥한 백그라운드 환경에서 수개월 동안 단 1초의 먹통 지연 시간도 없이 완벽한 자가 면역력을 유지하며 청정하게 살아 숨 쉬길 원한다면, 이 스튜어드와 워드 격리 디자인 패턴을 핵심 길목마다 장착하여 신뢰성 100%의 무결점 시스템 수평 확장을 이뤄내자.